CS162
Operating Systems and Systems Programming
Lecture 5

System Programming: Files, Sockets, Pipes

Professor Ion Stoica & Matei Zaharia

https://cs162.org/

Slides based on prior slide decks from David Culler, Natacha Crooks, John Kubiatowicz, Alison Norman and Lorenzo Alvisi

I/O and Storage Layers

Application / Service

High Level I/O — *Streams (buffered I/O)*

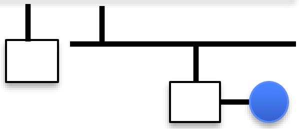
Low Level I/O — *File Descriptors*

Syscall — *open(), read(), write(), close(), …*

Open File Descriptions

File System — *Files/Directories/Indexes*

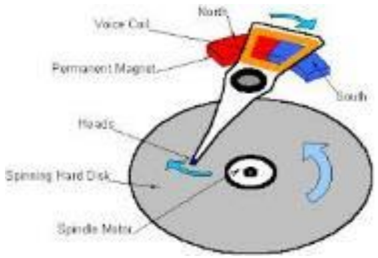
I/O Driver — *Commands and Data Transfers*



Disks, Flash, Controllers, DMA







Low-Level vs. High-Level File API

Low-Level Operation:

```
ssize_t read(…) {

    asm code … syscall # into %eax
    put args into registers %ebx, …
    special trap instruction
        Kernel:
          get args from regs
          dispatch to system func
          Do the work to read from the file
          Store return value in %eax
    get return values from regs

  Return data to caller

};
```

High-Level Operation:

```
ssize_t fread(…) {
  Check buffer for contents
  Return data to caller if available
    asm code … syscall # into %eax
    put args into registers %ebx, …
    special trap instruction
        Kernel:
          get args from regs
          dispatch to system func
          Do the work to read from the file
          Store return value in %eax
    get return values from regs
  Update buffer with excess data
  Return data to caller
};
```

High-Level vs. Low-Level File API

- Streams are buffered in user memory:

```
printf("Beginning of line ");
sleep(10); // sleep for 10 seconds
printf("and end of line\n");
```

  Prints out everything at once

- Operations on file descriptors are visible immediately

```
write(STDOUT_FILENO, "Beginning of line ", 18);
sleep(10);
write("and end of line \n", 16);
```

  Outputs "Beginning of line" 10 seconds earlier than “and end of line”

What’s below the surface ?

File descriptor number
- an int

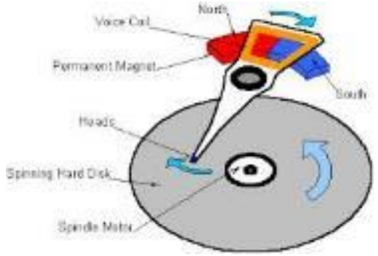
File description
- a struct with all the info about the files

Application / Service

Layer	
High Level I/O	streams
Low Level I/O	handles
Syscall	registers
File System	descriptions
I/O Driver	Commands and Data Transfers
	Disks, Flash, Controllers, DMA





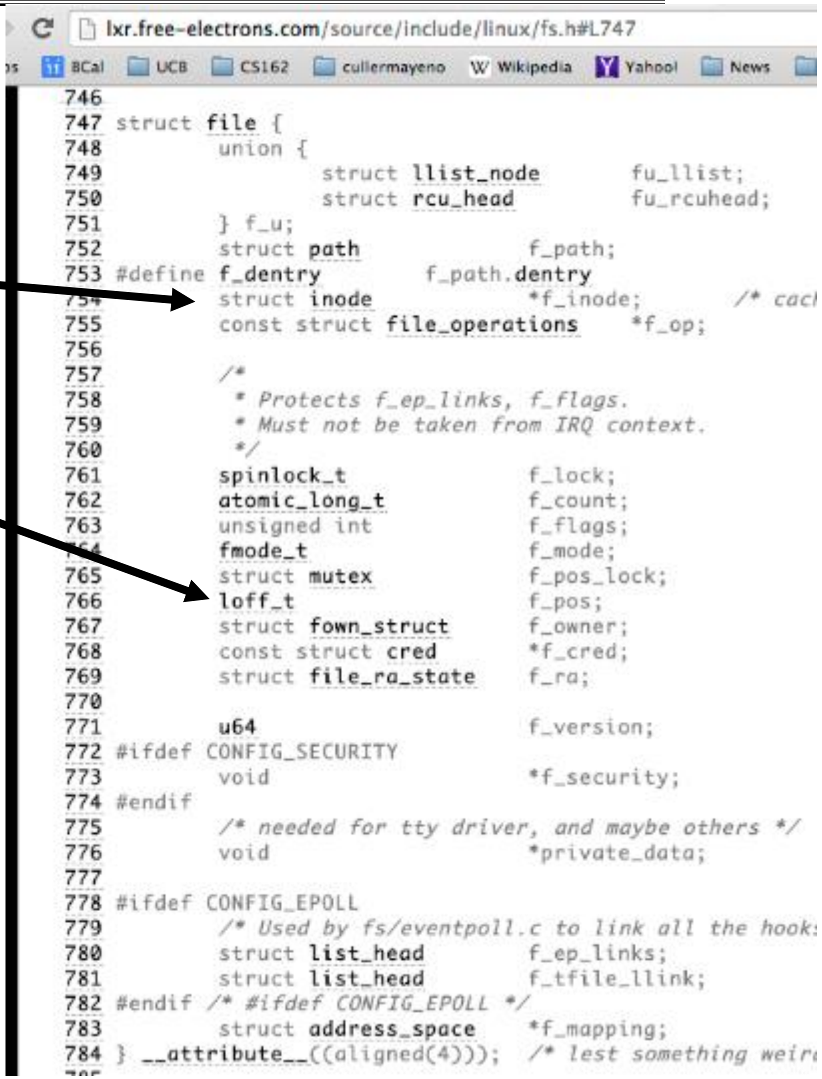


What's in an Open File Description?

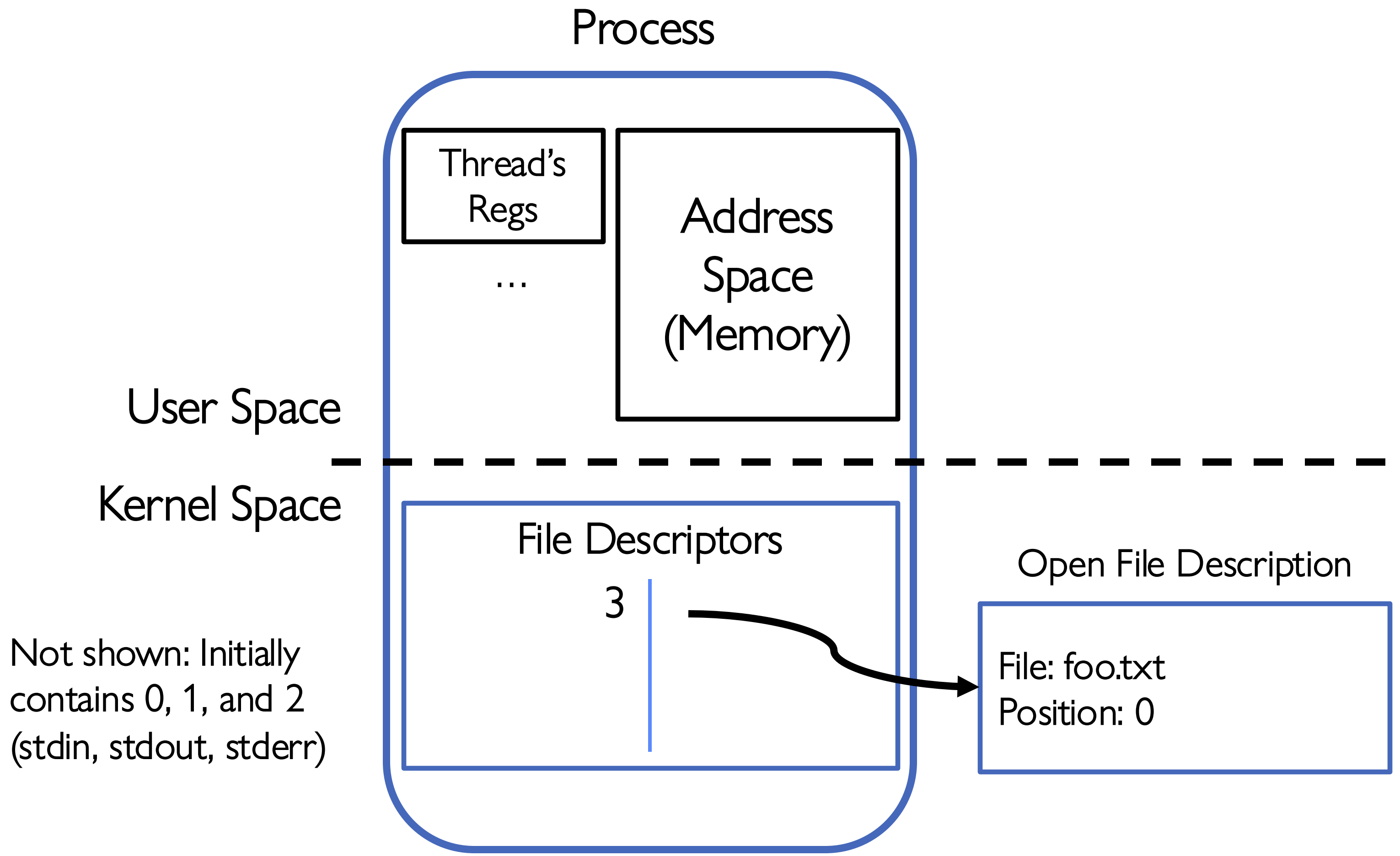
Inside Kernel!

For our purposes, the two most important things are:

- Where to find the file data on disk
- The current position within the file

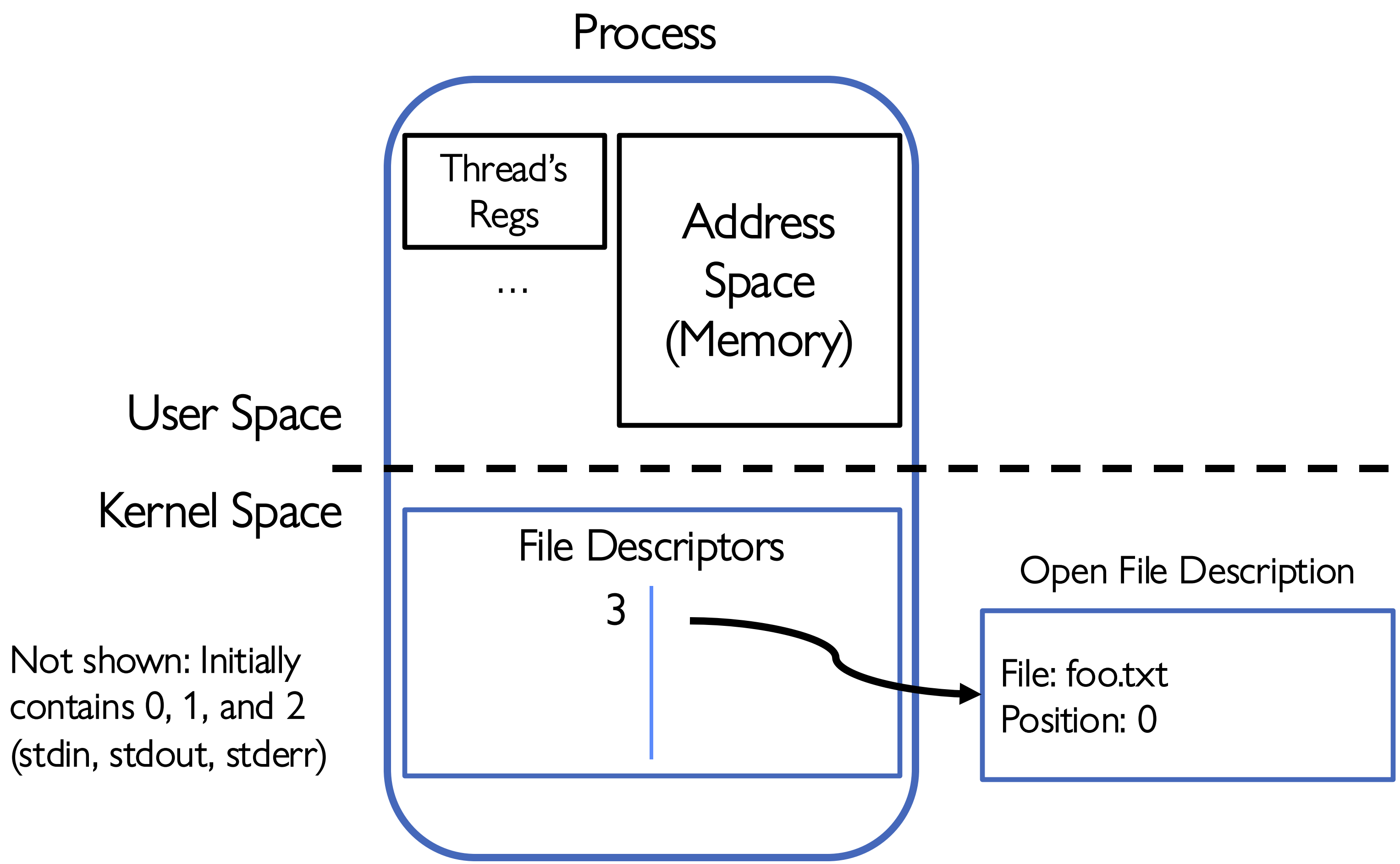


Process-Specific File Descriptor Table inside Kernel



Suppose that we execute
open("foo.txt")
and that the result is 3

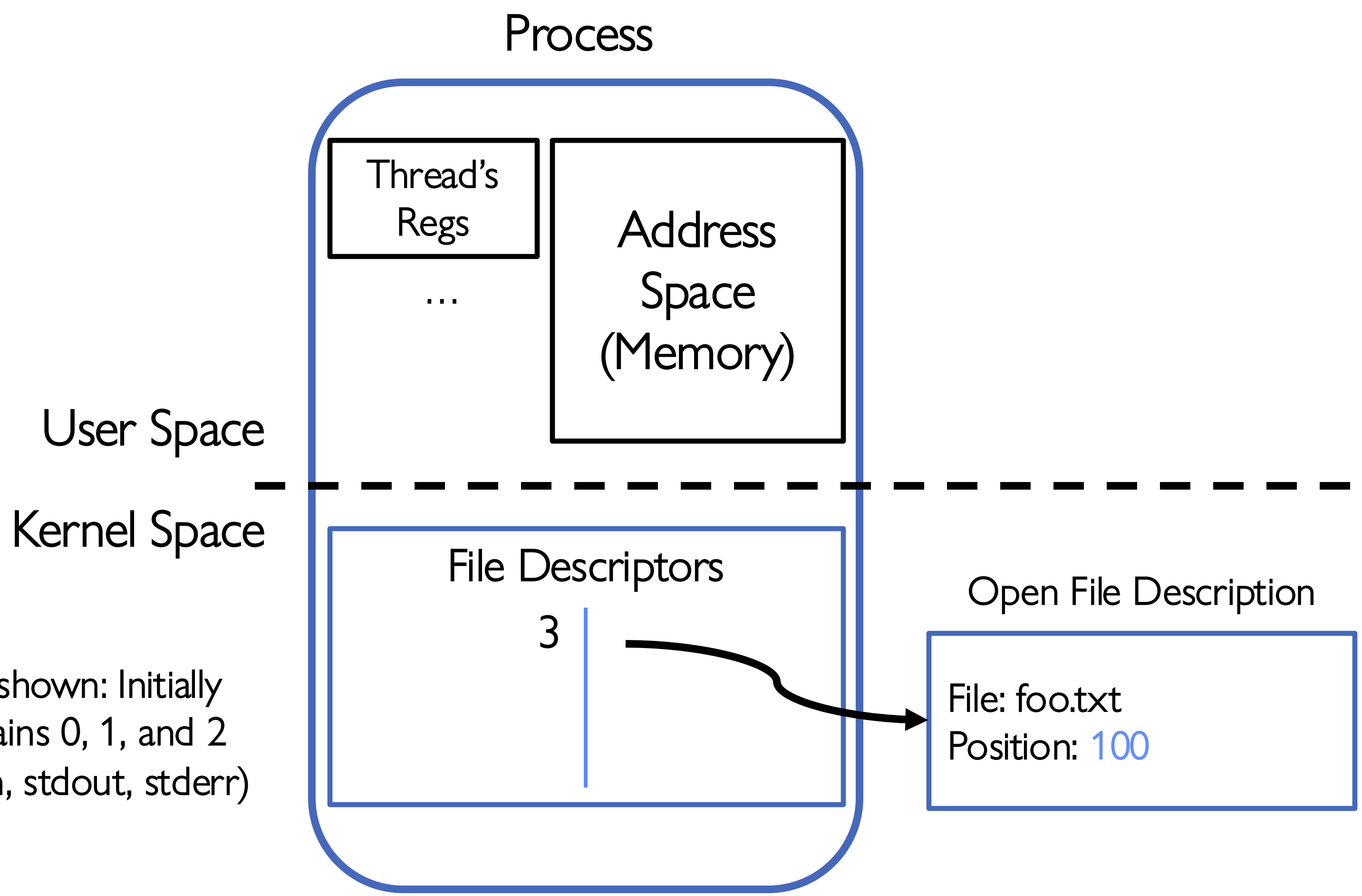
Process-Specific File Descriptor Table inside Kernel



Suppose that we execute
open("foo.txt")
and that the result is 3

Next, suppose that we execute
read(3, buf, 100)
and that the result is 100

Process-Specific File Descriptor Table inside Kernel

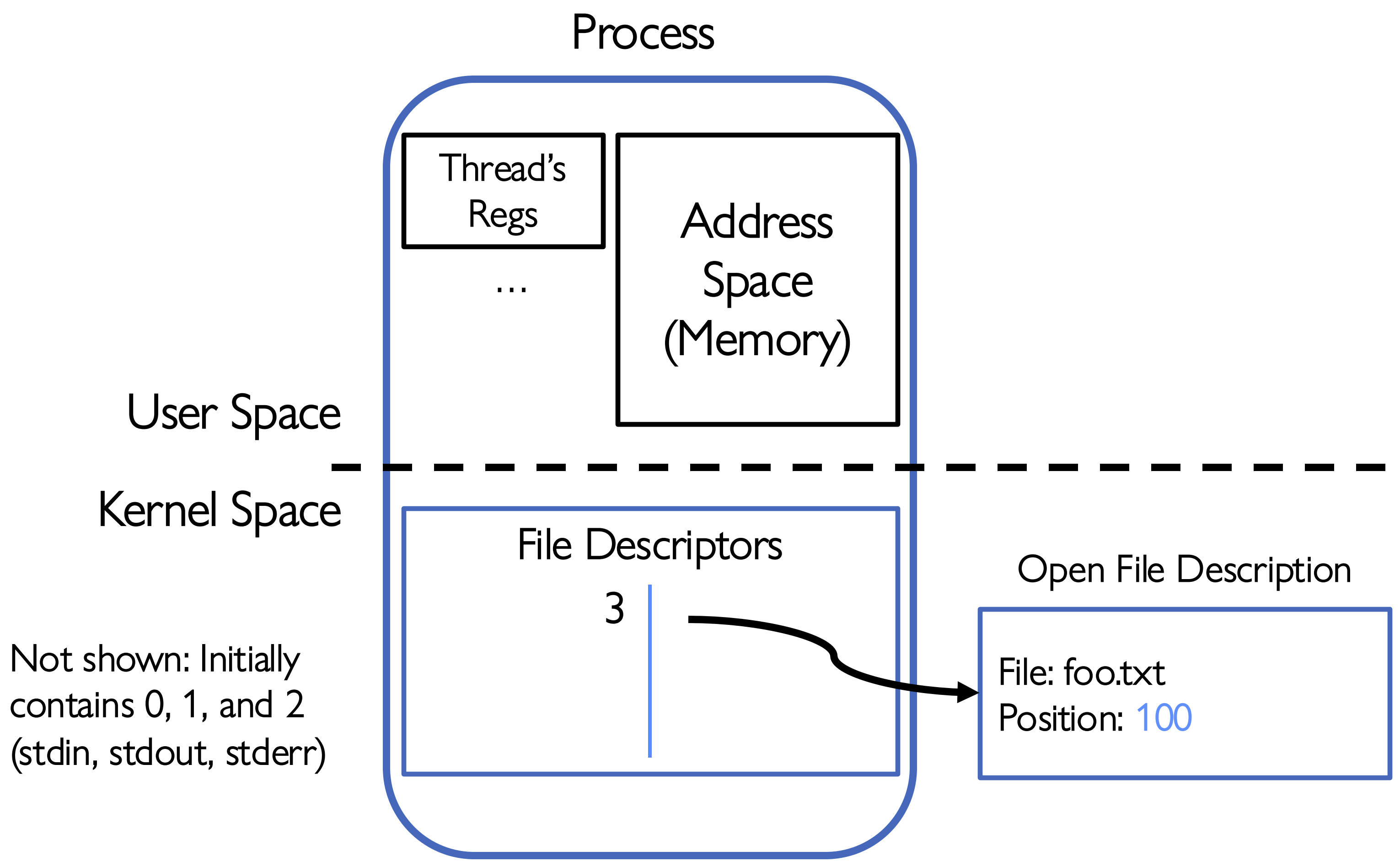


Not shown: Initially contains 0, 1, and 2 (stdin, stdout, stderr)

Suppose that we execute
open("foo.txt")
and that the result is 3

Next, suppose that we execute
read(3, buf, 100)
and that the result is 100

Process-Specific File Descriptor Table inside Kernel

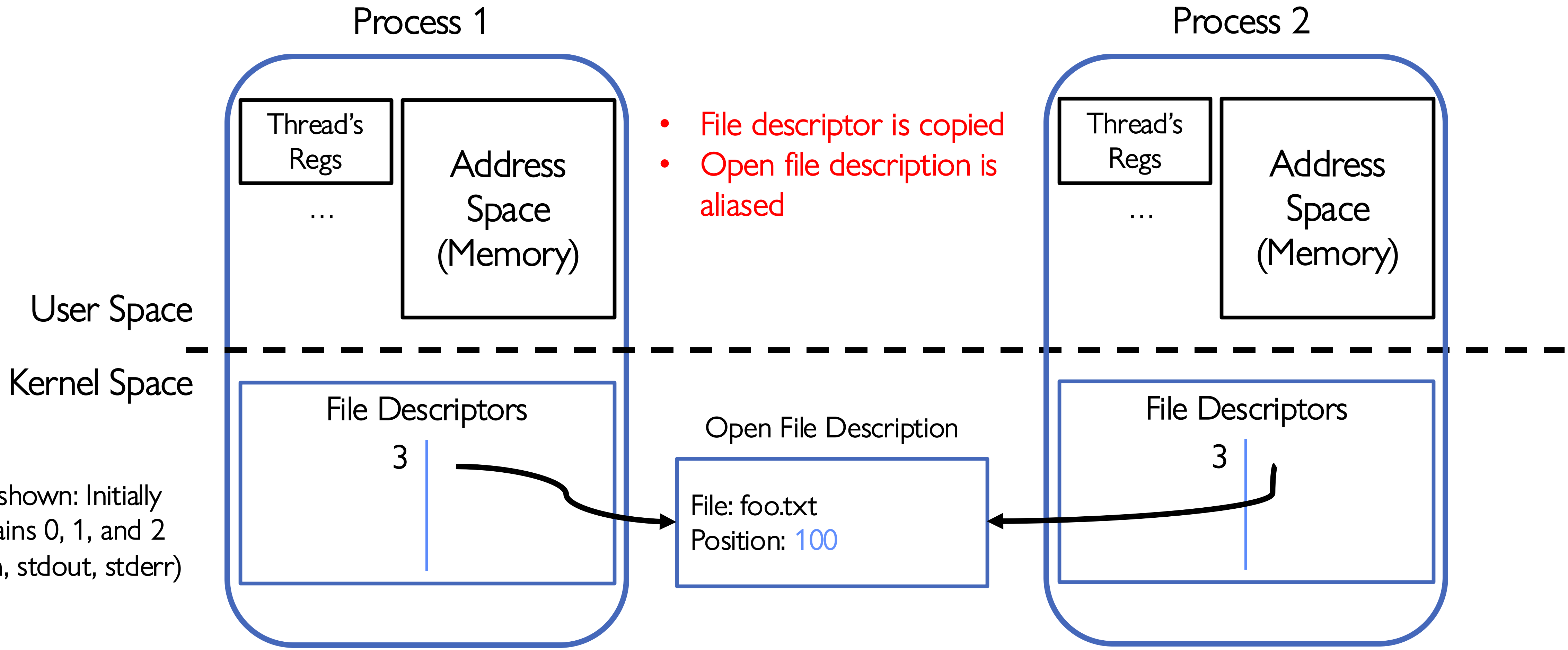


Suppose that we execute
open("foo.txt")
and that the result is 3

Next, suppose that we execute
read(3, buf, 100)
and that the result is 100

Finally, suppose that we execute
close(3)

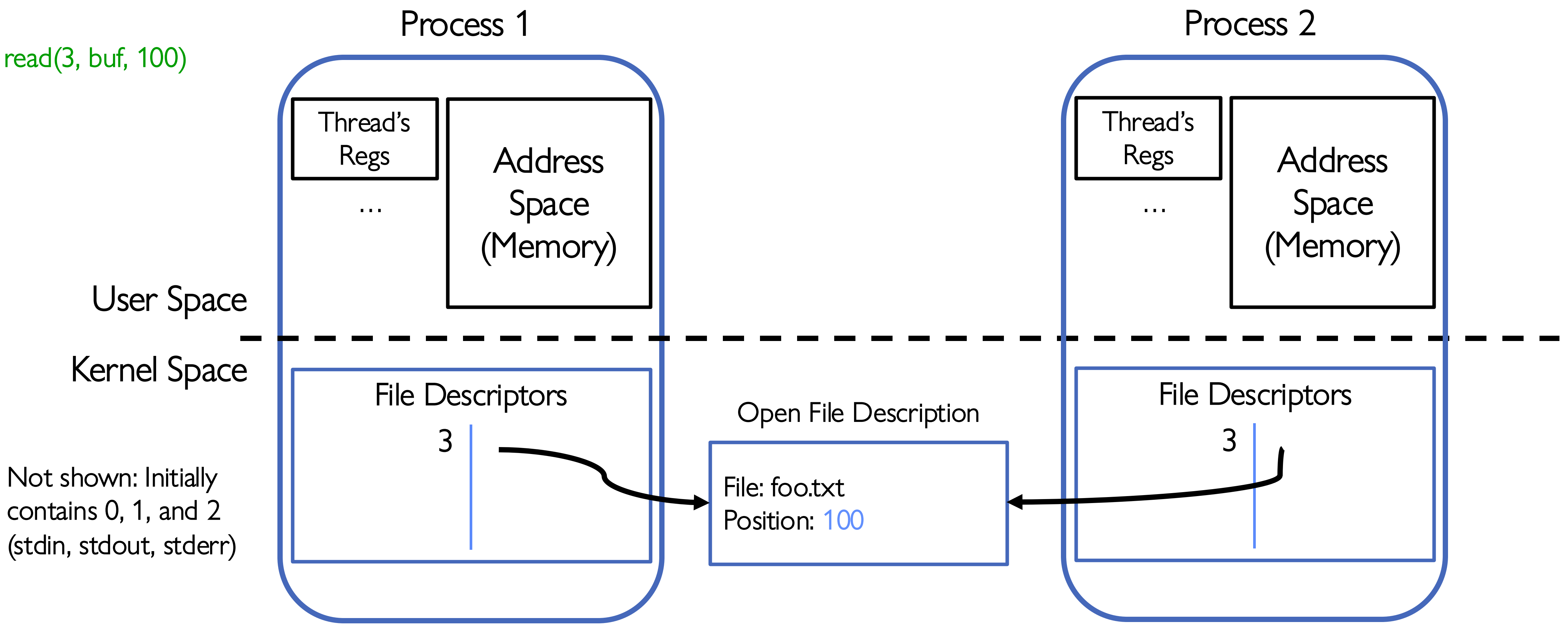
Instead of Closing, let's `fork()`!



- File descriptor is copied
- Open file description is aliased

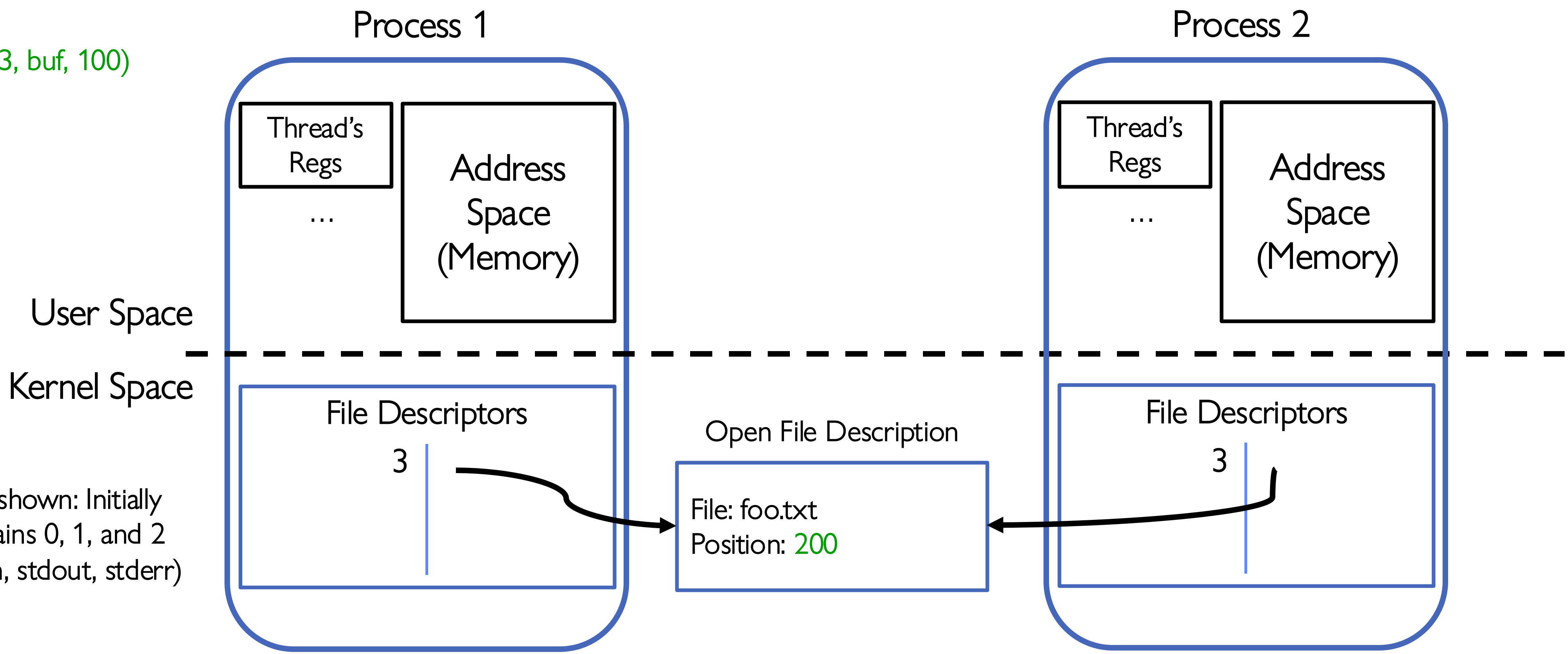
Not shown: Initially contains 0, 1, and 2 (stdin, stdout, stderr)

Open File Description is *Aliased*



Open File Description is *Aliased*

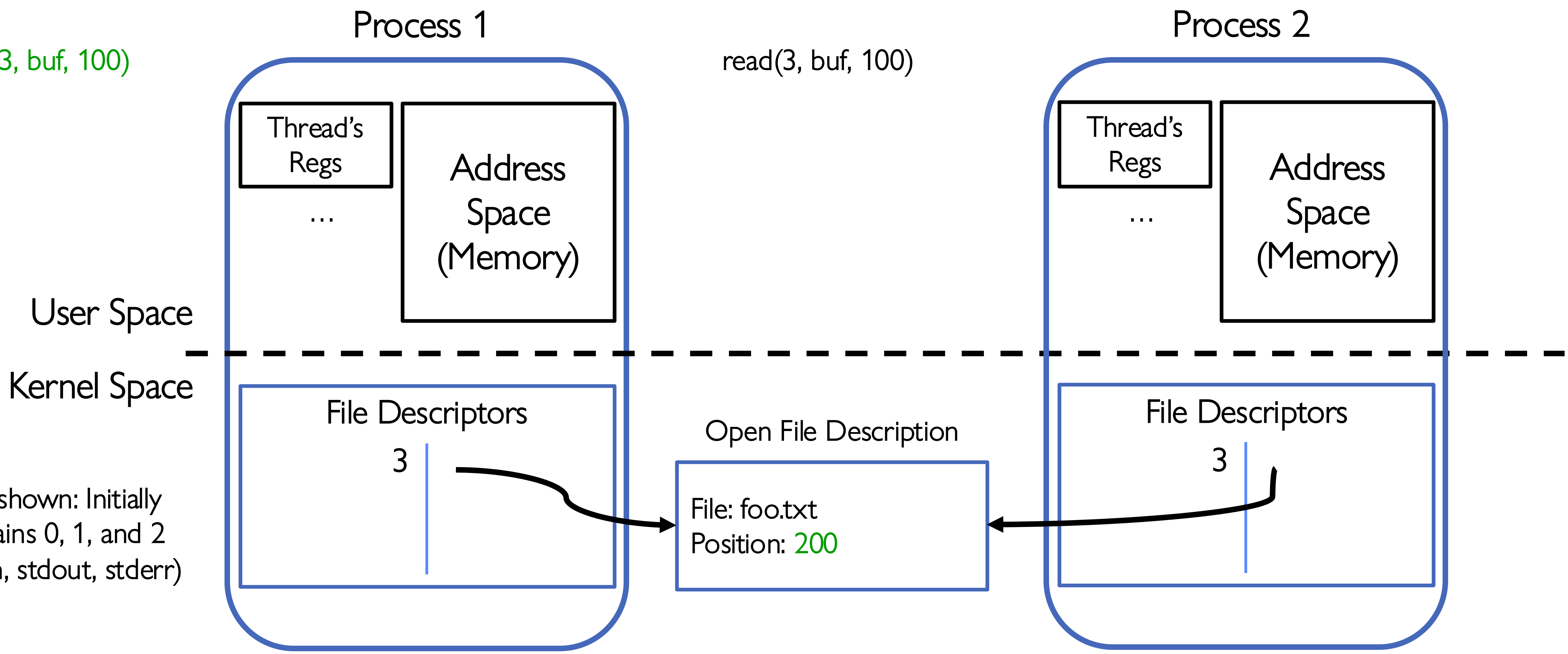
read(3, buf, 100)



Not shown: Initially contains 0, 1, and 2 (stdin, stdout, stderr)

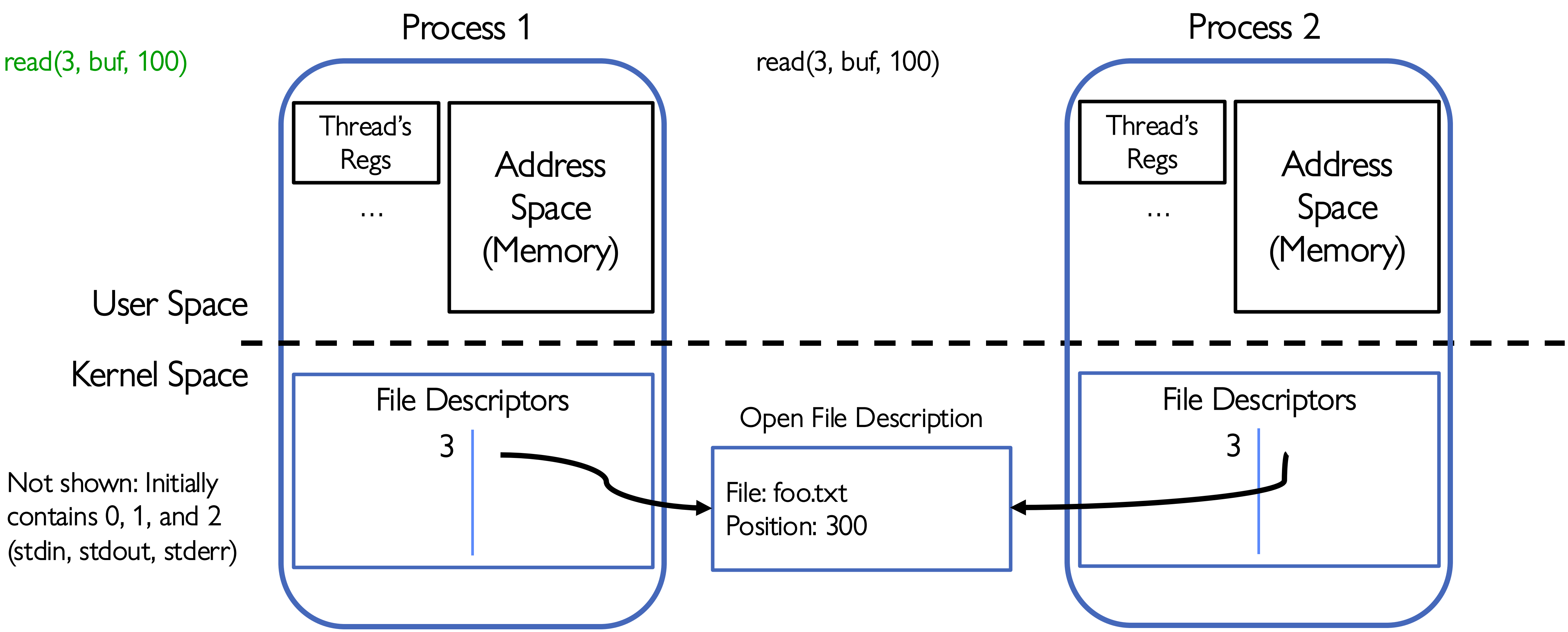
Open File Description is *Aliased*

read(3, buf, 100)



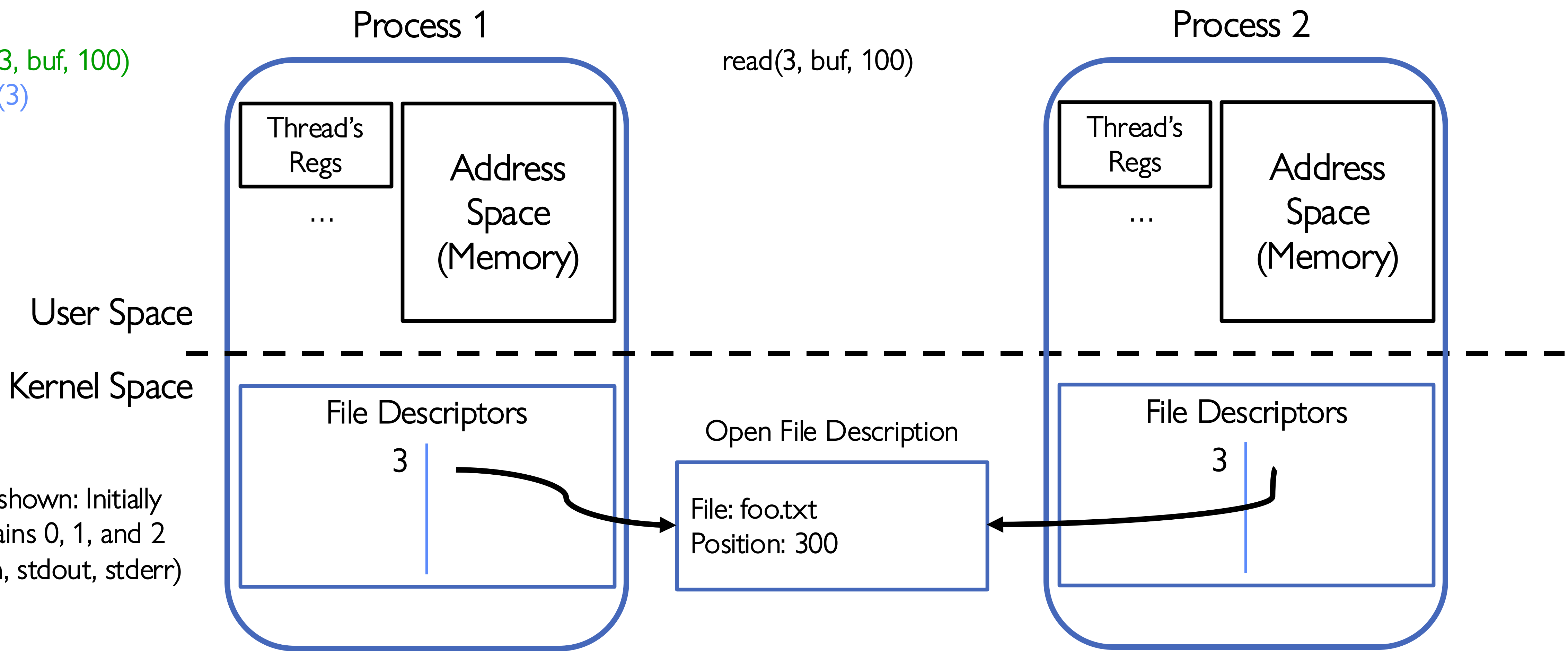
Not shown: Initially contains 0, 1, and 2 (stdin, stdout, stderr)

Open File Description is *Aliased*



File Descriptor is *Copied*

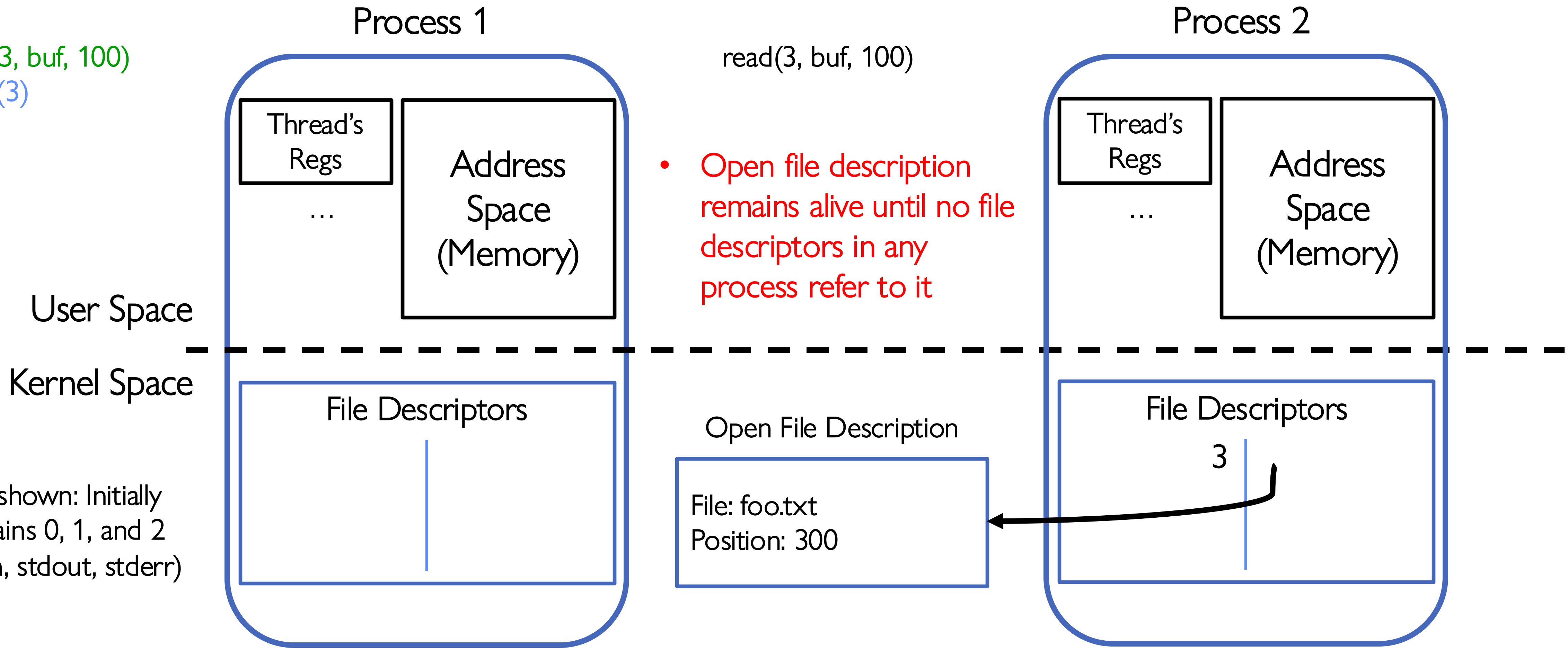
read(3, buf, 100)
close(3)



Not shown: Initially contains 0, 1, and 2 (stdin, stdout, stderr)

File Descriptor is *Copied*

read(3, buf, 100)
close(3)



- Open file description remains alive until no file descriptors in any process refer to it

Not shown: Initially contains 0, 1, and 2 (stdin, stdout, stderr)

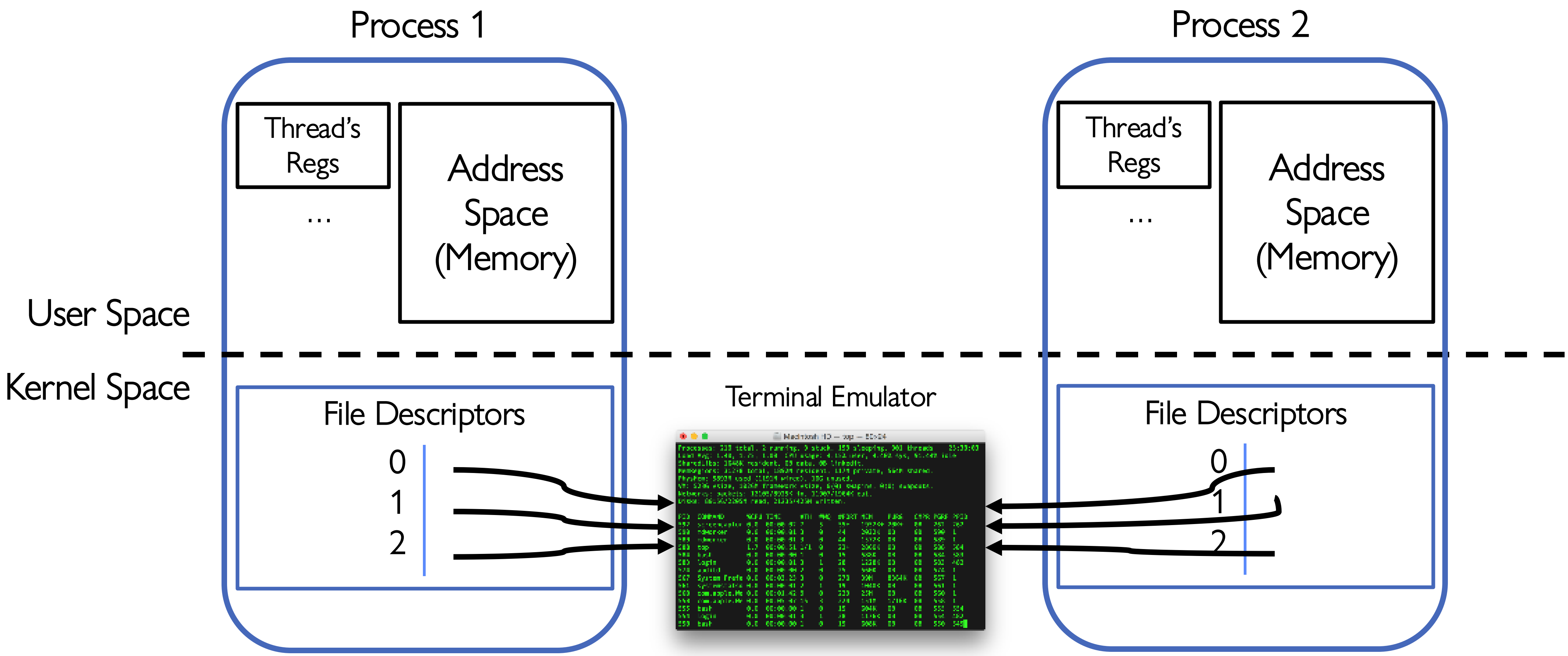
Why is Aliasing the Open File Description a Good Idea?

- It allows for *shared resources* between processes

Example: Shared Terminal Emulator

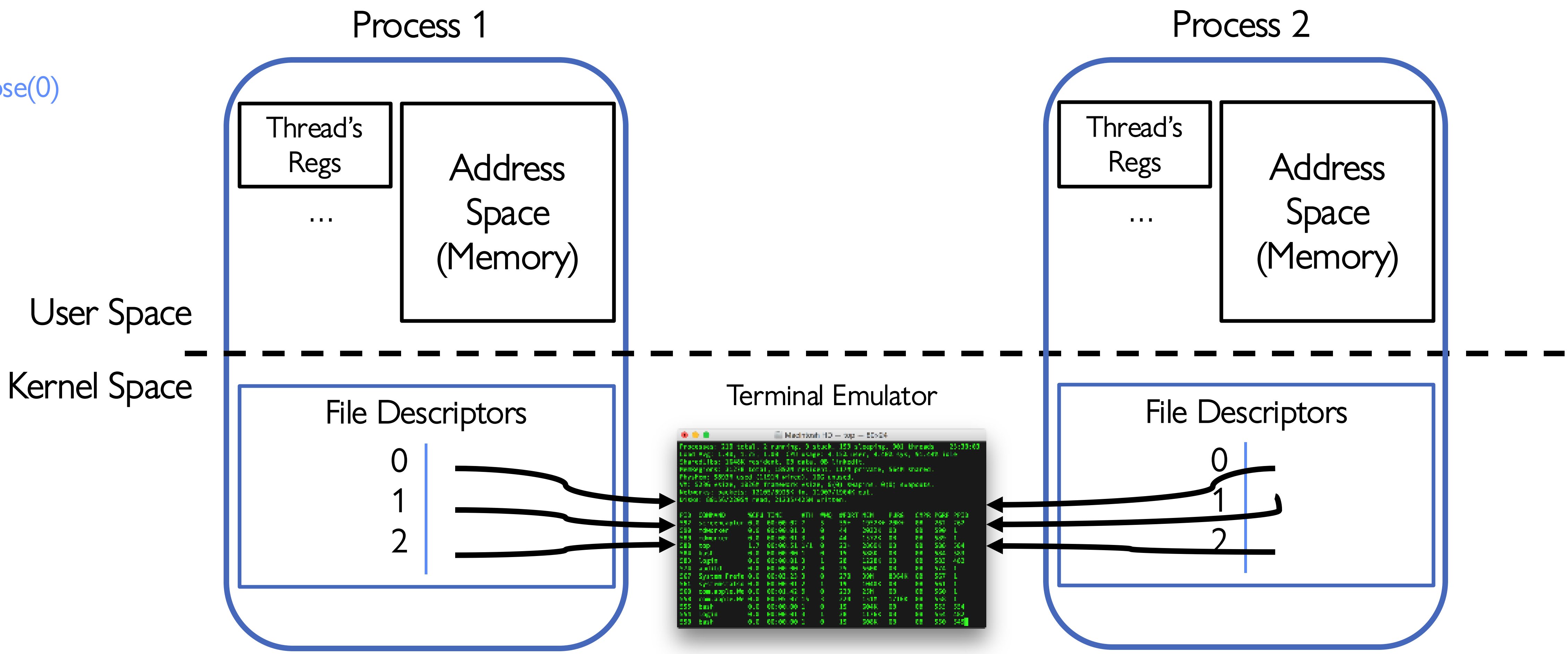
- When you `fork()` a process, the parent's and child's `printf` outputs go to the same terminal

Example: Shared Terminal Emulator



Example: Shared Terminal Emulator

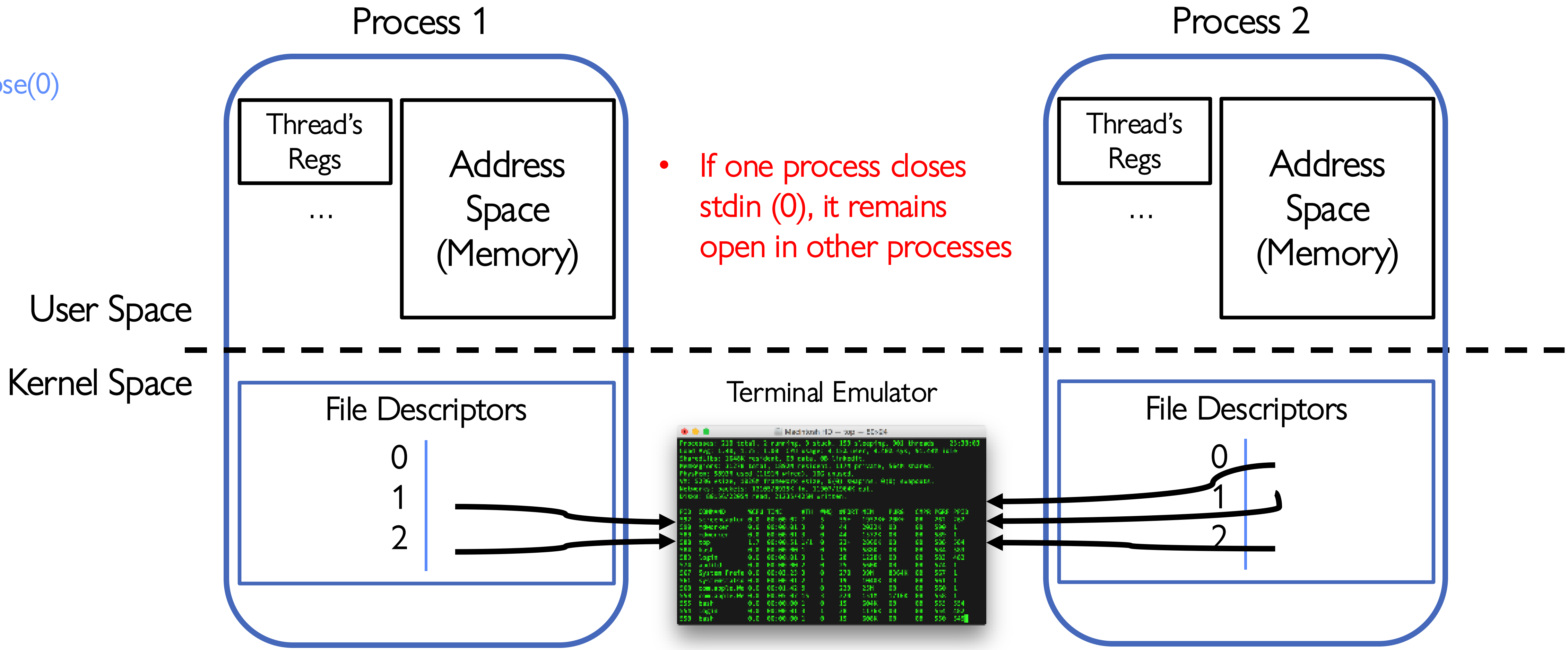
close(0)



Example: Shared Terminal Emulator

close(0)

- If one process closes stdin (0), it remains open in other processes

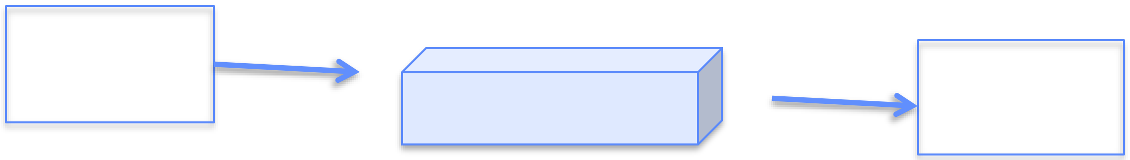


Inter-Process Communication (IPC)

Communication between processes

- Can we view files as communication channels?

```
write(wfd, wbuf, wlen);
```

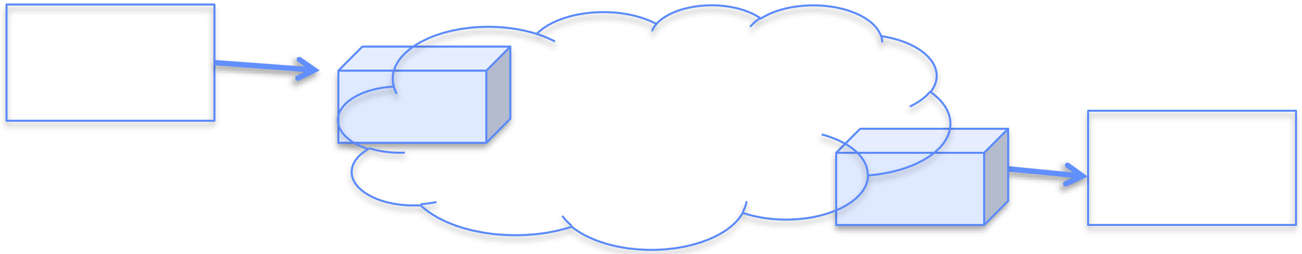


```
n = read(rfd,rbuf,rmax);
```

- Producer and Consumer of a file may be distinct processes
  – May be separated in time (or not)
- However, what if data written once and consumed once?
  – Don’t we want something more like a queue?
  – Can still look like File I/O!

Communication Across the world looks like file IO!

```
write(wfd, wbuf, wlen);
```



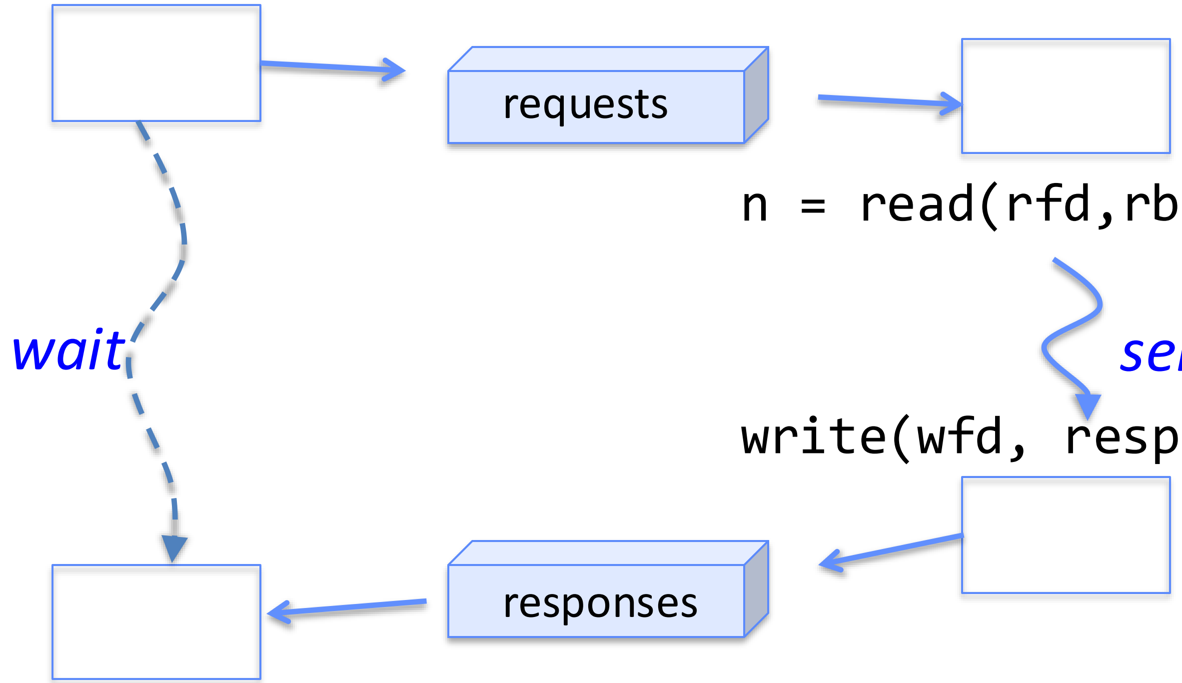
```
n = read(rfd,rbuf,rmax);
```

- Connected queues over the Internet
  - But what's the analog of open?
  - What is the namespace?
  - How are they connected in time?

Request Response Protocol

Client (issues requests) | **Server (performs operations)**

```
write(rqfd, rqbuf, buflen);
```



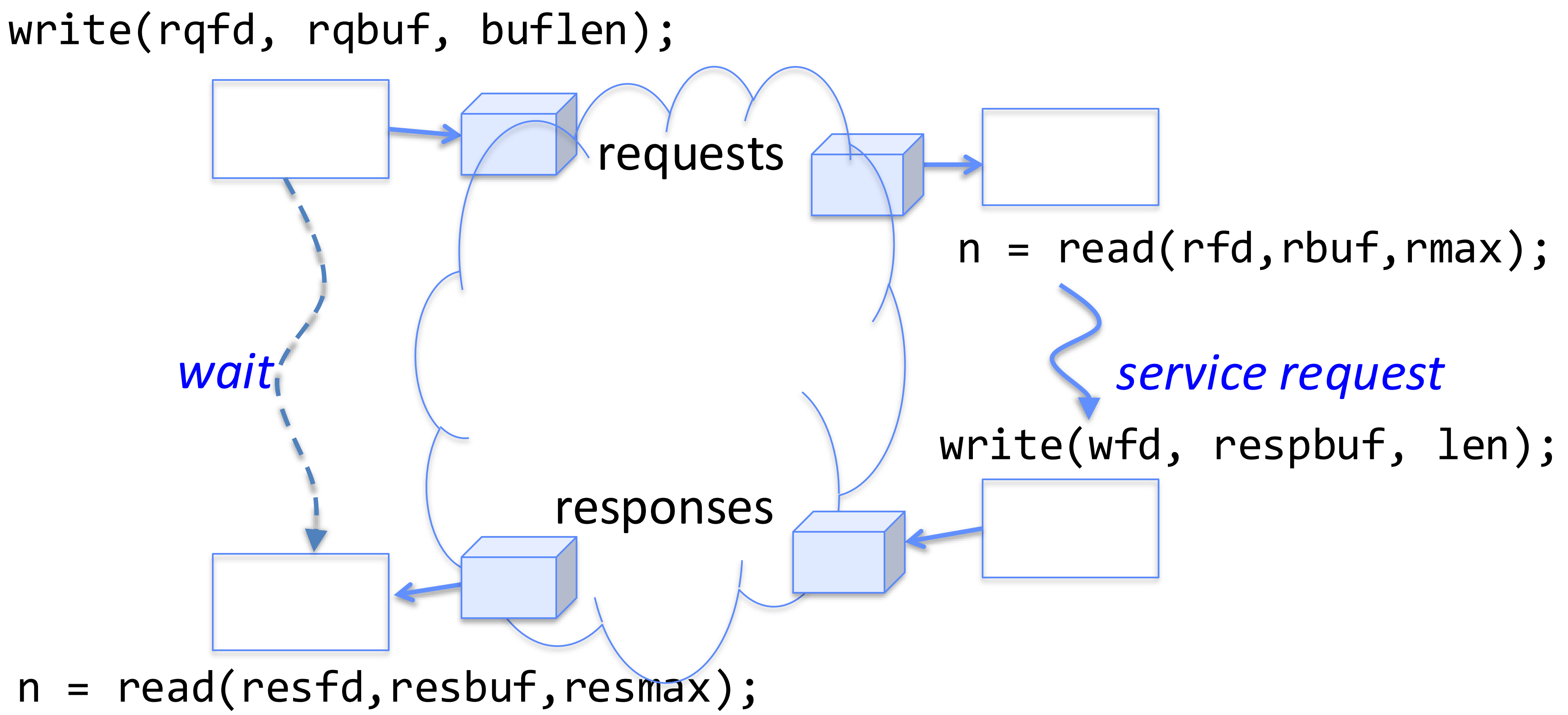

```
n = read(rfd,rbuf,rmax);
```

```
write(wfd, respbuf, len);
```

```
n = read(resfd,resbuf,resmax);
```

Request Response Protocol: Across Network

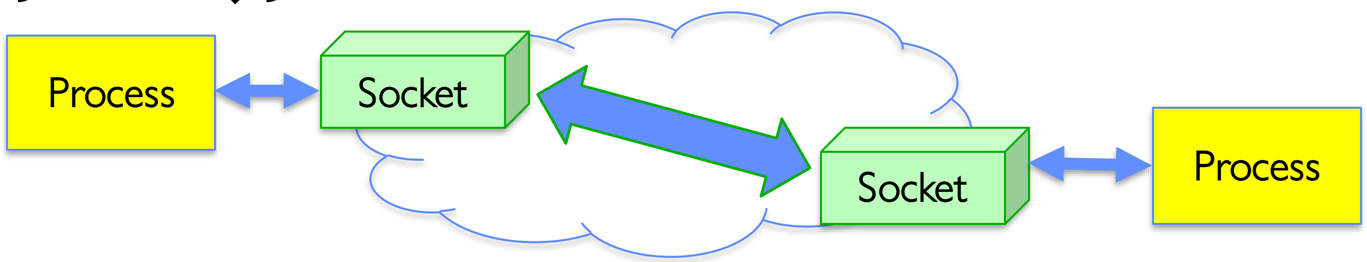
Client (issues requests) Server (performs operations)



The Socket Abstraction: Endpoint for Communication

- **Key Idea:** Communication across the world looks like File I/O

```
write(wfd, wbuf, wlen);
```




```
n = read(rfd, rbuf, rmax);
```

- Sockets: Endpoint for Communication
  – Queues to temporarily hold results
- Connection: Two Sockets Connected Over the network ⇒ IPC over network!
  – How to **open()**?
  – What is the namespace?
  – How are they connected in time?

Sockets: More Details

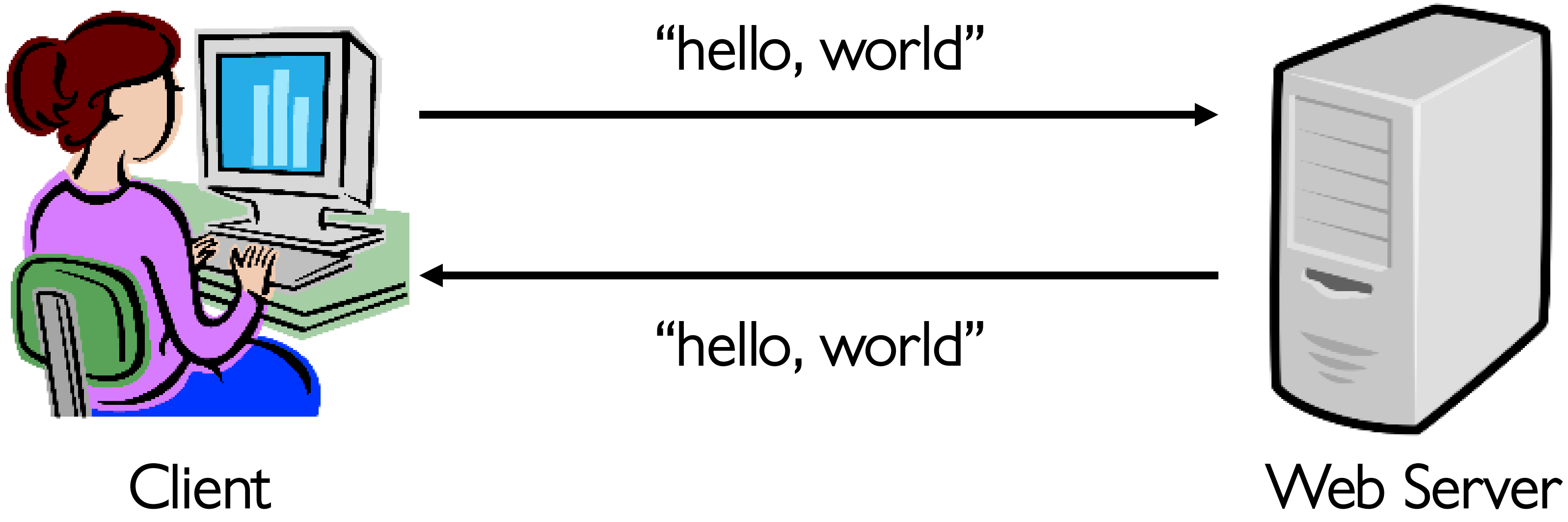
- **Socket:** An abstraction for one endpoint of a network connection
  - Another mechanism for **inter-process communication**
  - Most operating systems (Linux, Mac OS X, Windows) provide this, even if they don't copy rest of UNIX I/O
  - Standardized by POSIX
- First introduced in 4.2 BSD (Berkeley Standard Distribution) Unix
  - This release had some huge benefits (and excitement from potential users)
  - Runners waiting at release time to get release on tape and take to businesses
- Same abstraction for any kind of network
  - Local (within same machine)
  - The Internet (TCP/IP, UDP/IP)
  - Things "no one" uses anymore (OSI, Appletalk, IPX, …)

Sockets: More Details

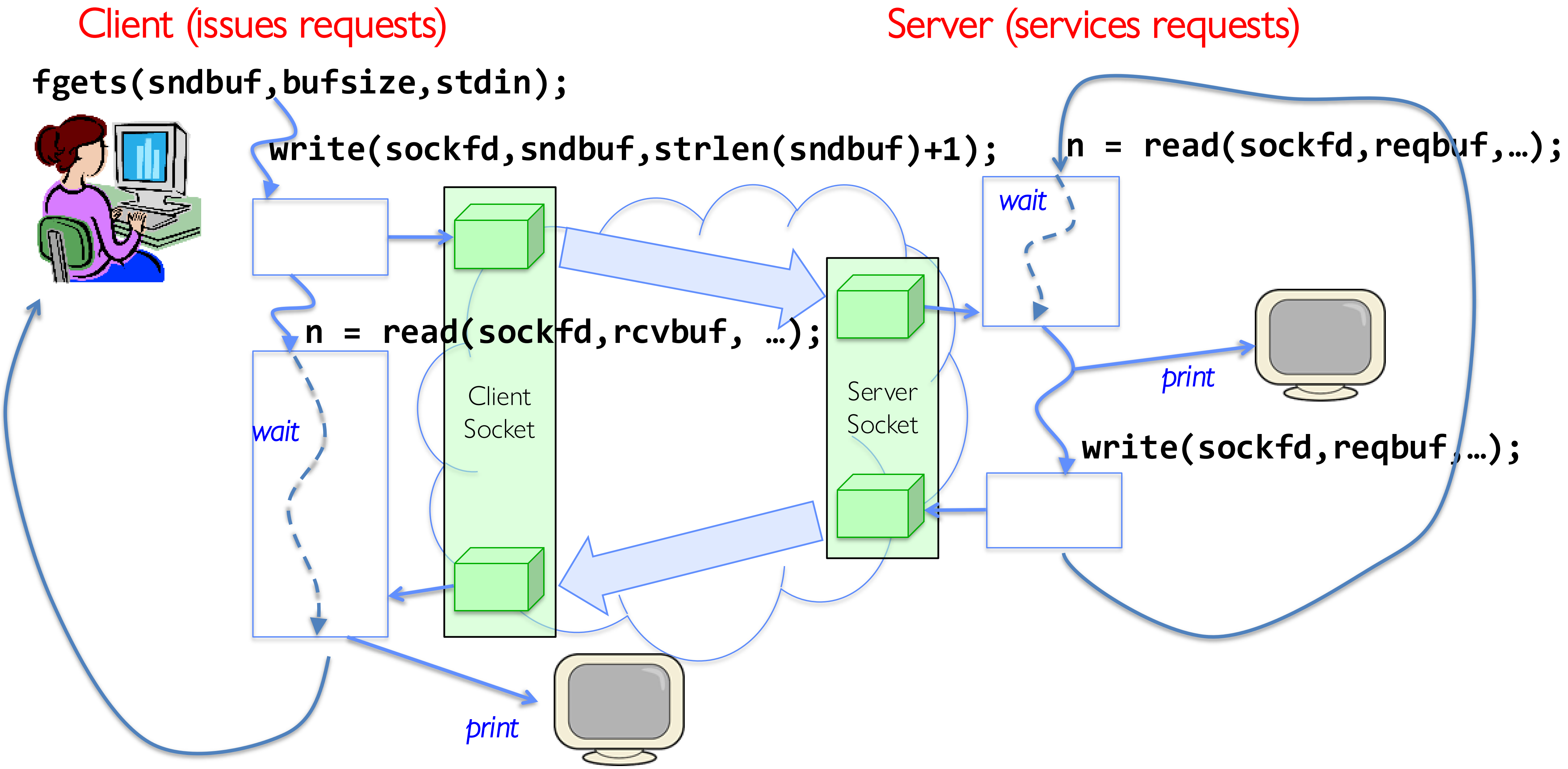
- Looks just like a file with a **file descriptor**
  - Corresponds to a network connection (*two* queues)
  - **`write`** adds to output queue (queue of data destined for other side)
  - **`read`** removes from it input queue (queue of data destined for this side)
  - Some operations do not work, e.g. **`lseek`**

- How can we use sockets to support real applications?
  - A bidirectional byte stream isn't useful on its own…
  - May need messaging facility to partition stream into chunks
  - May need RPC facility to translate one environment to another and provide the abstraction of a function call over the network

Simple Example: Echo Server



Simple Example: Echo Server



Echo client-server example

```
void client(int sockfd) {
  int n;
  char sndbuf[MAXIN]; char rcvbuf[MAXOUT];
  while (1) {
    fgets(sndbuf,MAXIN,stdin);                  /* prompt */
    write(sockfd, sndbuf, strlen(sndbuf)+1); /* send (including null terminator) */
    memset(rcvbuf,0,MAXOUT);                    /* clear */
    n=read(sockfd, rcvbuf, MAXOUT);             /* receive */
    write(STDOUT_FILENO, rcvbuf, n);            /* echo */
}
```

```
void server(int consockfd) {
  char reqbuf[MAXREQ];
  int n;
  while (1) {
    memset(reqbuf,0, MAXREQ);
    n = read(consockfd,reqbuf,MAXREQ); /* Recv */
    if (n <= 0) return;
    write(STDOUT_FILENO, reqbuf, n);
    write(consockfd, reqbuf, n); /* echo*/
  }
}
```

What Assumptions are we Making?

- Reliable
  - Write to a file => Read it back. Nothing is lost.
  - Write to a (TCP) socket => Read from the other side, same.

- In order (sequential stream)
  - Write X then write Y => read gets X then read gets Y

- When ready?
  - File read gets whatever is there at the time
    - » Actually, need to loop and read until we receive the terminator ('\0')
  - Assumes writing already took place
  - Blocks if nothing has arrived yet

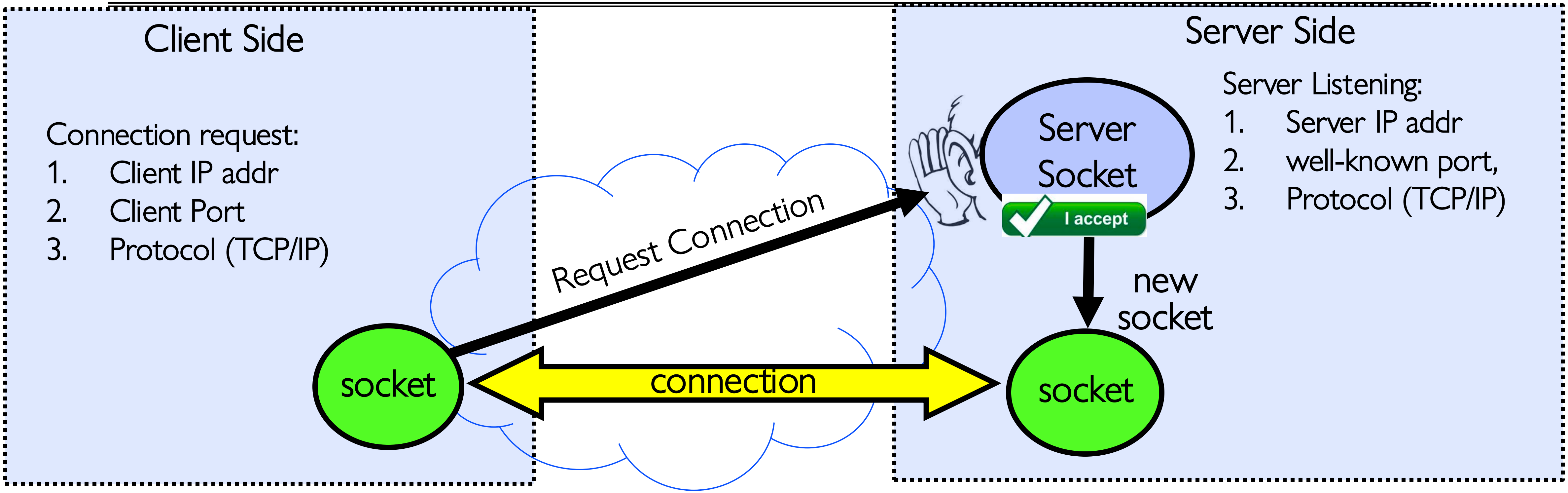
Socket Creation

- File systems provide a collection of permanent objects in a structured name space:
  - Processes open, read/write/close them
  - Files exist independently of processes
  - Easy to name what file to **`open()`**
- Pipes: one-way communication between processes on same (physical) machine (see later)
  - Single queue
  - Created transiently by a call to **`pipe()`**
  - Passed from parent to children (descriptors inherited from parent process)
- Sockets: two-way communication between processes on same or different machine
  - Two queues (one in each direction)
  - Processes can be on separate machines: no common ancestor
  - How do we *name* the objects we are **opening**?
  - How do these completely independent programs know that the other wants to "talk" to them?

Namespaces for Communication over IP

- Hostname
  - www.eecs.berkeley.edu
- IP address
  - 128.32.244.172 (IPv4, 32-bit Integer)
  - 2607:f140:0:81::f (IPv6, 128-bit Integer)
- Port Number
  - 0-1023 are "well known" or "system" ports
    - » E.g., 80 (http), 443 (secure web)
    - » Superuser privileges to bind to one
  - 1024 – 49151 are "registered" ports (registry)
    - » Assigned by IANA for specific services
  - 49152–65535 ($2^{15}+2^{14}$ to $2^{16}-1$) are "dynamic" or "private"
    - » Automatically allocated as "ephemeral ports"

Connection Setup over TCP/IP



- Special kind of socket: **server socket**
  - Has file descriptor
  - Can't read or write
- Two operations:
  1. **listen()**: Start allowing clients to connect
  2. **accept()**: Create a *new socket* for a *particular* client

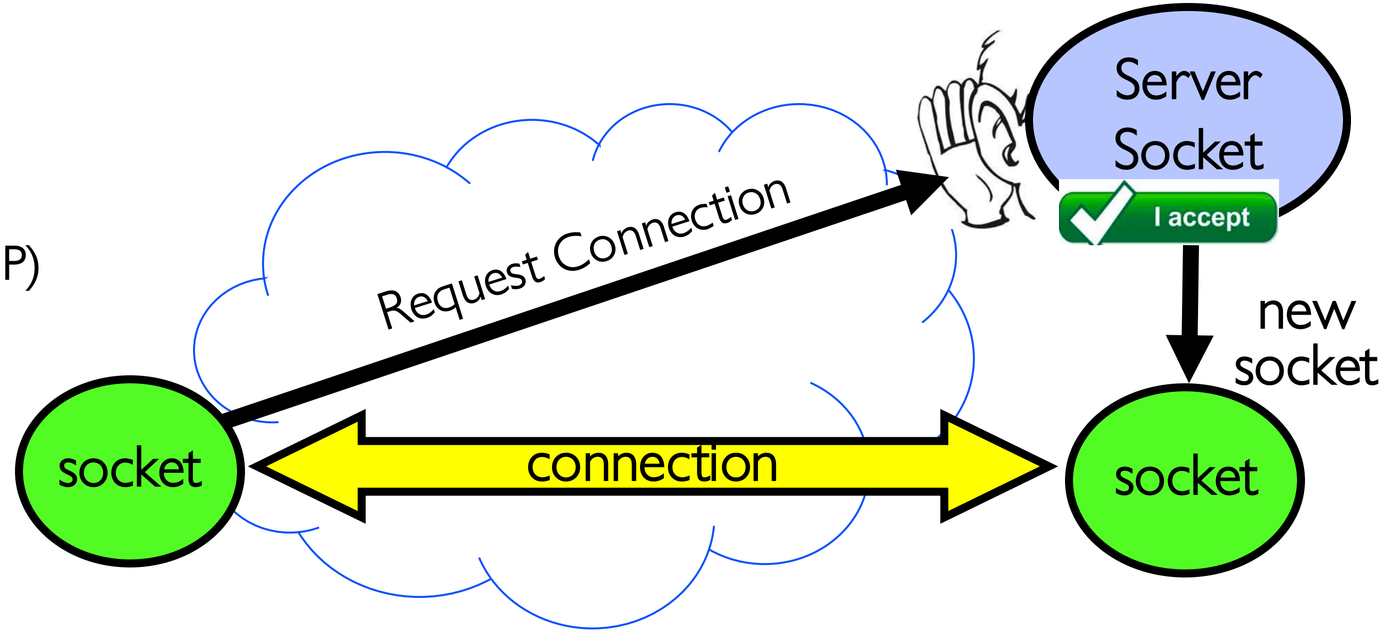
Connection Setup over TCP/IP

Client Side

Connection request:
1. Client IP addr
2. Client Port
3. Protocol (TCP/IP)

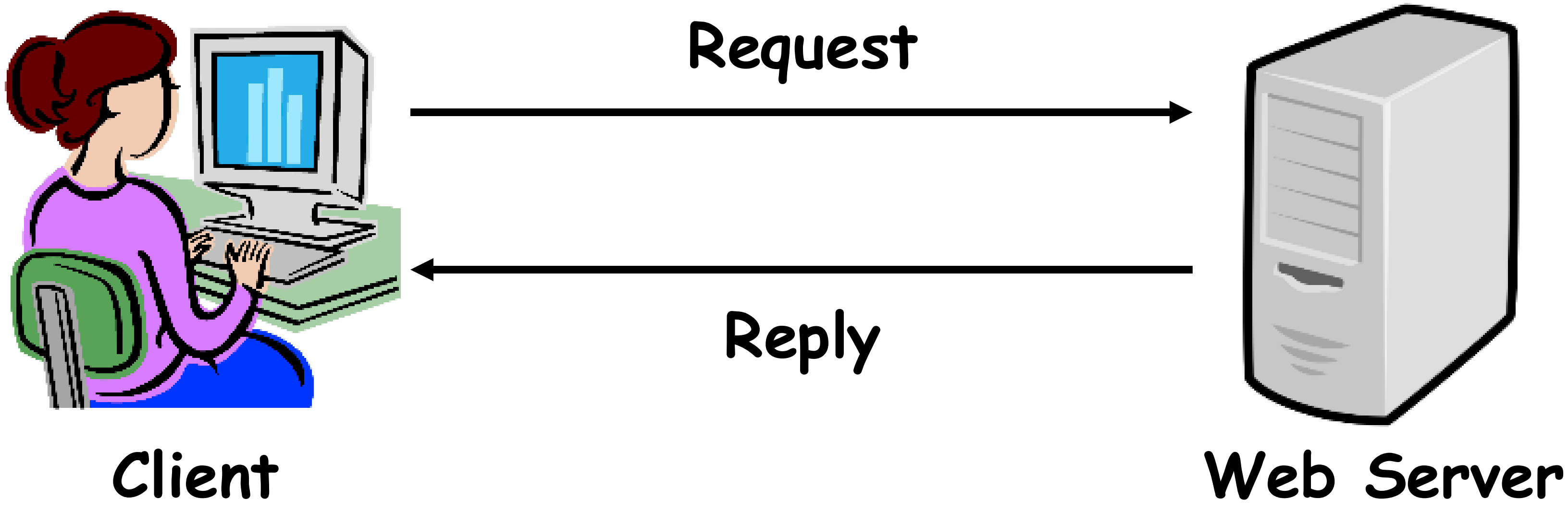
Server Side

Server Listening:
1. Server IP addr
2. well-known port,
3. Protocol (TCP/IP)



- 5-Tuple identifies each connection:
  1. Source IP Address
  2. Destination IP Address
  3. Source Port Number
  4. Destination Port Number
  5. Protocol (always TCP here)
- Often, Client Port "randomly" assigned
  – Done by OS during client socket setup
- Server Port often "well known"
  – 80 (web), 443 (secure web), 25 (sendmail), etc
  – Well-known ports from 0—1023

Web Server

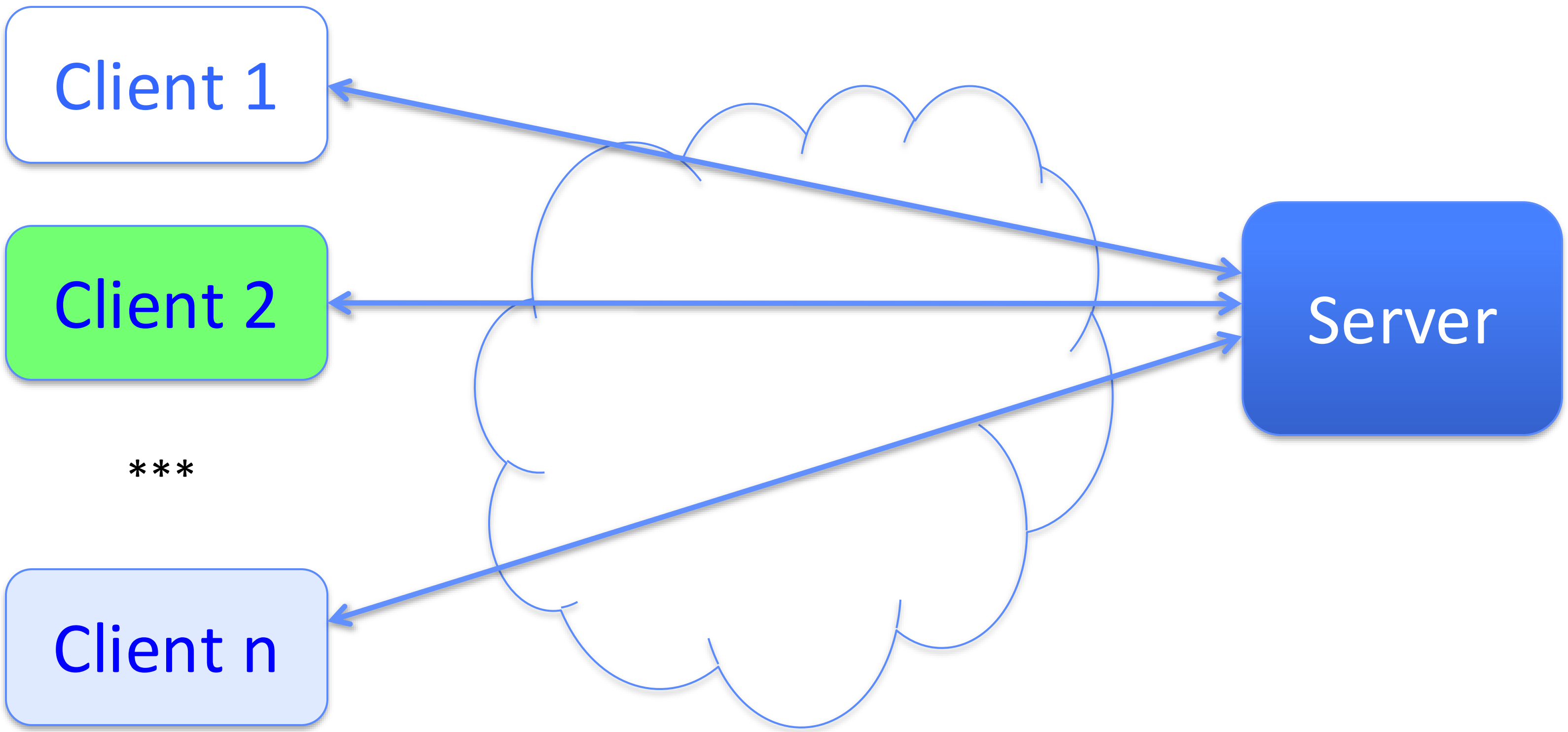


Administrivia

- By now everyone should be part of a 4 people group
  - If not, please do this as soon as possible.

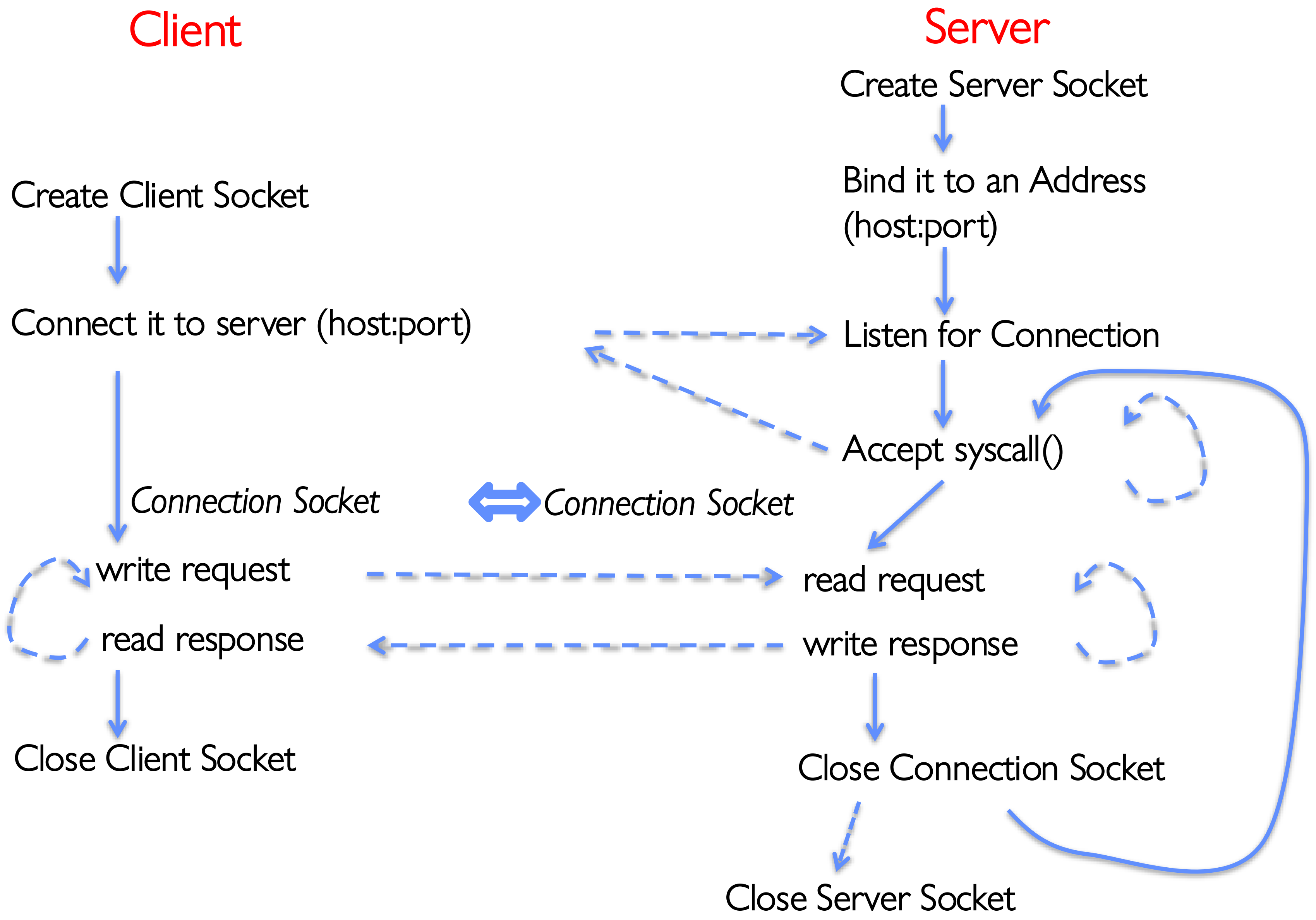
- First midterm: Wednesday, October 1
  - During the class
  - Lecture on 9/29 is included
  - Review session on 9/30 (TBA)
  - Cheat sheet: hand-written, two side A4 paper (or smaller)

Client-Server Models



- File servers, web, FTP, Databases, …
- Many clients accessing a common server

Simple Web Server



Client Code

```
char *host_name, *port_name;

// Create a socket
struct addrinfo *server = lookup_host(host_name, port_name);
int sock_fd = socket(server->ai_family, server->ai_socktype,
                     server->ai_protocol);

// Connect to specified host and port
connect(sock_fd, server->ai_addr, server->ai_addrlen);

// Carry out Client-Server protocol
run_client(sock_fd);

/* Clean up on termination */
close(sock_fd);
```

Client-Side: Getting the Server Address

```
struct addrinfo *lookup_host(char *host_name, char *port_name) {
  struct addrinfo *server;
  struct addrinfo hints;
  memset(&hints, 0, sizeof(hints));
  hints.ai_family = AF_UNSPEC;         /* Includes AF_INET and AF_INET6 */
  hints.ai_socktype = SOCK_STREAM;     /* Essentially TCP/IP */

  int rv = getaddrinfo(host_name, port_name, &hints, &server);
  if (rv != 0) {
    printf("getaddrinfo failed: %s\n", gai_strerror(rv));
    return NULL;
  }
  return server;
}
```

Server Code (v1)

```
// Create socket to listen for client connections
char *port_name;
struct addrinfo *server = setup_address(port_name);
int server_socket = socket(server->ai_family,
                           server->ai_socktype, server->ai_protocol);
// Bind socket to specific port
bind(server_socket, server->ai_addr, server->ai_addrlen);
// Start listening for new client connections
listen(server_socket, MAX_QUEUE);

while (1) {
  // Accept a new client connection, obtaining a new socket
  int conn_socket = accept(server_socket, NULL, NULL);
  serve_client(conn_socket);
  close(conn_socket);
}
close(server_socket);
```

Server Address: Itself (wildcard IP), Passive

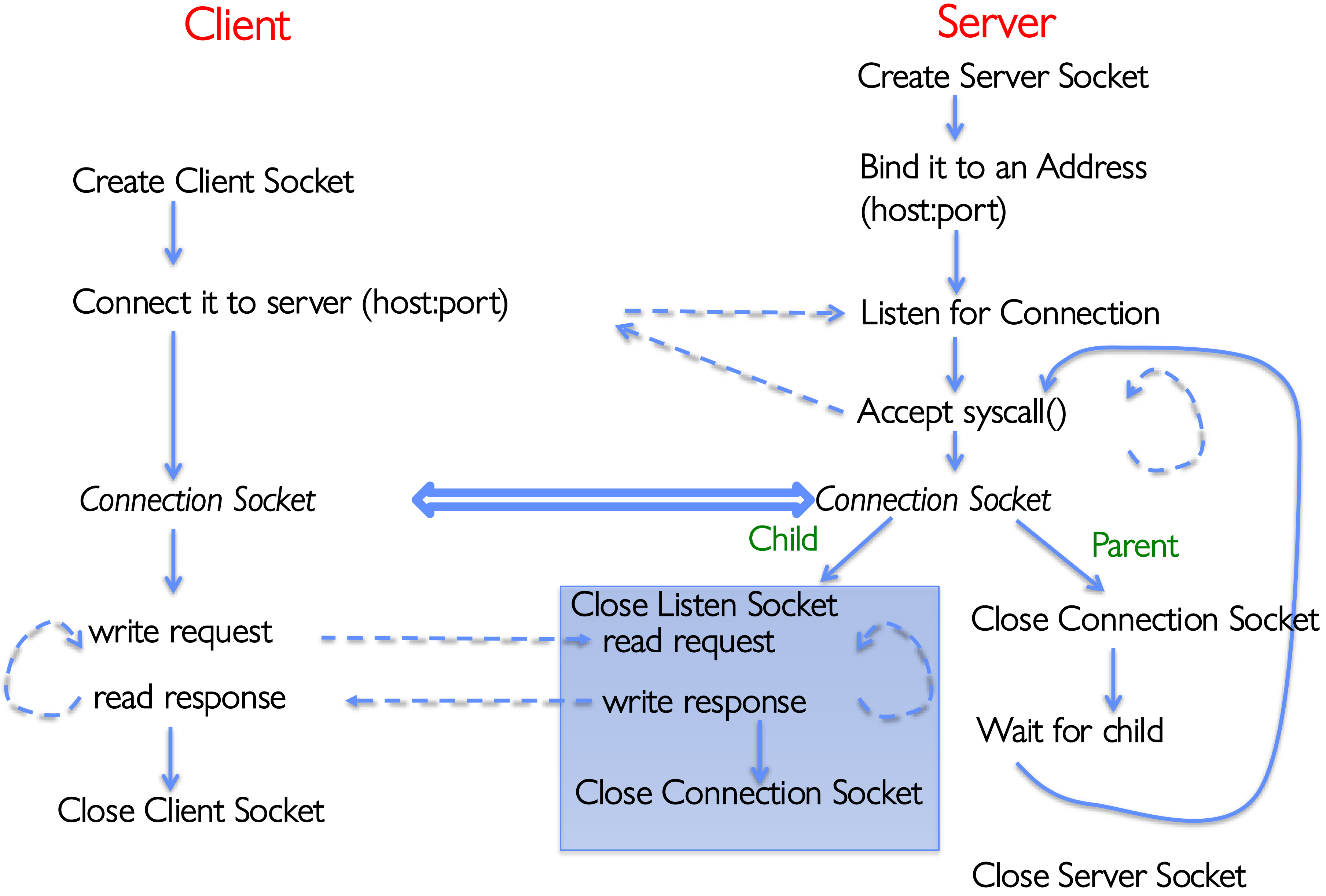
```
struct addrinfo *setup_address(char *port) {
  struct addrinfo *server;
  struct addrinfo hints;
  memset(&hints, 0, sizeof(hints));
  hints.ai_family = AF_UNSPEC;           /* Includes AF_INET and AF_INET6 */
  hints.ai_socktype = SOCK_STREAM;       /* Essentially TCP/IP */
  hints.ai_flags = AI_PASSIVE;           /* Set up for server socket */

  int rv = getaddrinfo(NULL, port, &hints, &server);  /* No address! (any local IP) */
  if (rv != 0) {
    printf("getaddrinfo failed: %s\n", gai_strerror(rv));
    return NULL;
  }
  return server;
}
```

- Accepts any connections on the specified port

How Could the Server Protect Itself?

- Handle each connection in a separate process
  – This will mean that the logic serving each request will be "sandboxed" away from the main server process
- In the following code, keep in mind:
  – `fork()` will duplicate *all* of the parent's file descriptors (i.e. pointers to sockets!)
  – We keep control over accepting new connections in the parent
  – New child connection for each remote client

Server With Protection (each connection has own process)



Server Code (v2)

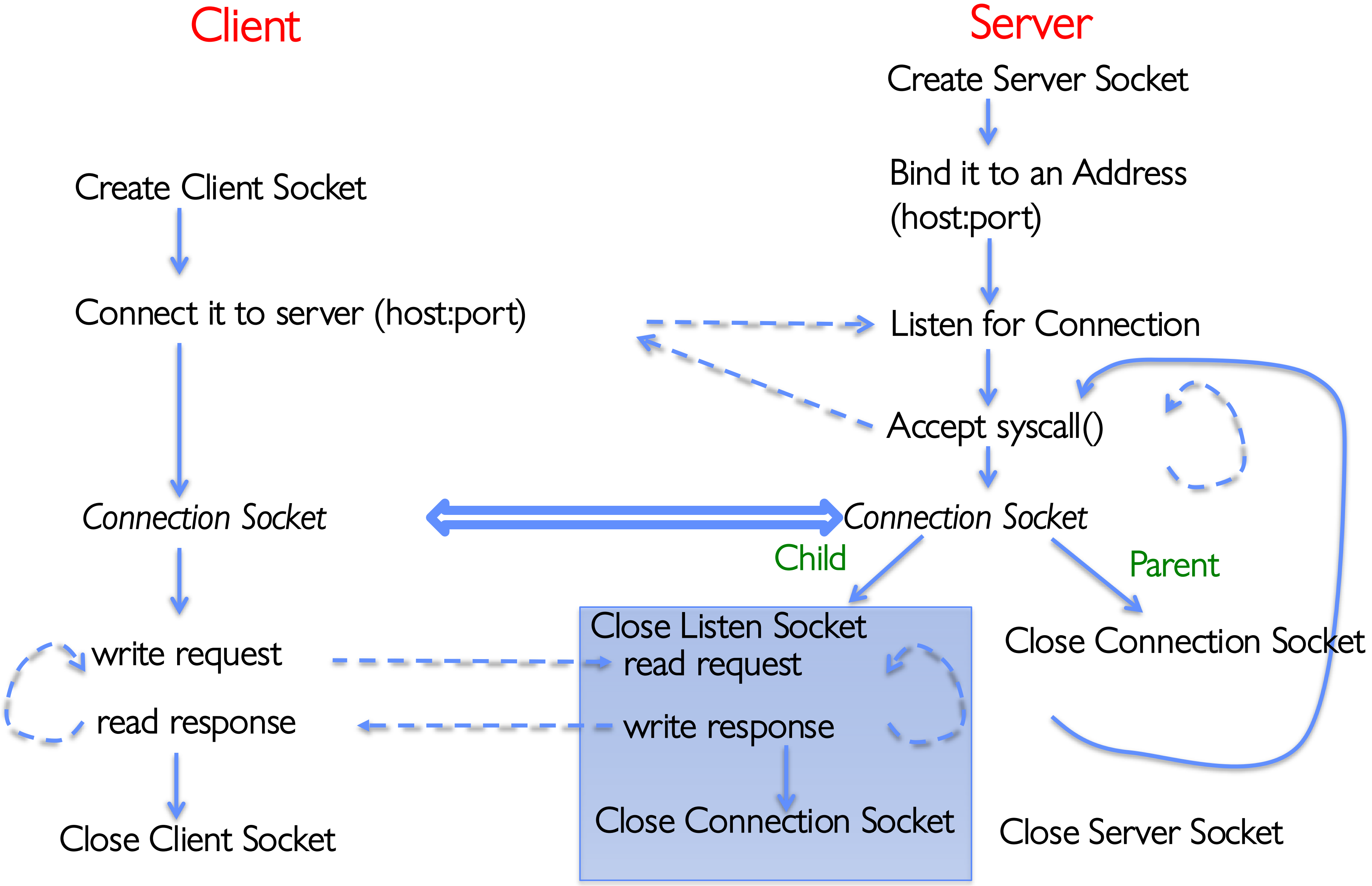
```
// Socket setup code elided…
listen(server_socket, MAX_QUEUE);
while (1) {
  // Accept a new client connection, obtaining a new socket
  int conn_socket = accept(server_socket, NULL, NULL);
  pid_t pid = fork();
  if (pid == 0) {
    close(server_socket);
    serve_client(conn_socket);
    close(conn_socket);
    exit(0);
  } else {
    close(conn_socket);
    wait(NULL);
  }
}
close(server_socket);
```

How to make a Concurrent Server

- So far, in the server:
  - Listen will queue requests
  - Buffering present elsewhere
  - But server *waits* for each connection to terminate before servicing the next
    - » This is the standard shell pattern

- A concurrent server can handle and service a new connection before the previous client disconnects
  - Simple – just don't wait in parent!
  - Perhaps not so simple – multiple child processes better not have data races with one another through file system/etc!

Server With Protection and Concurrency



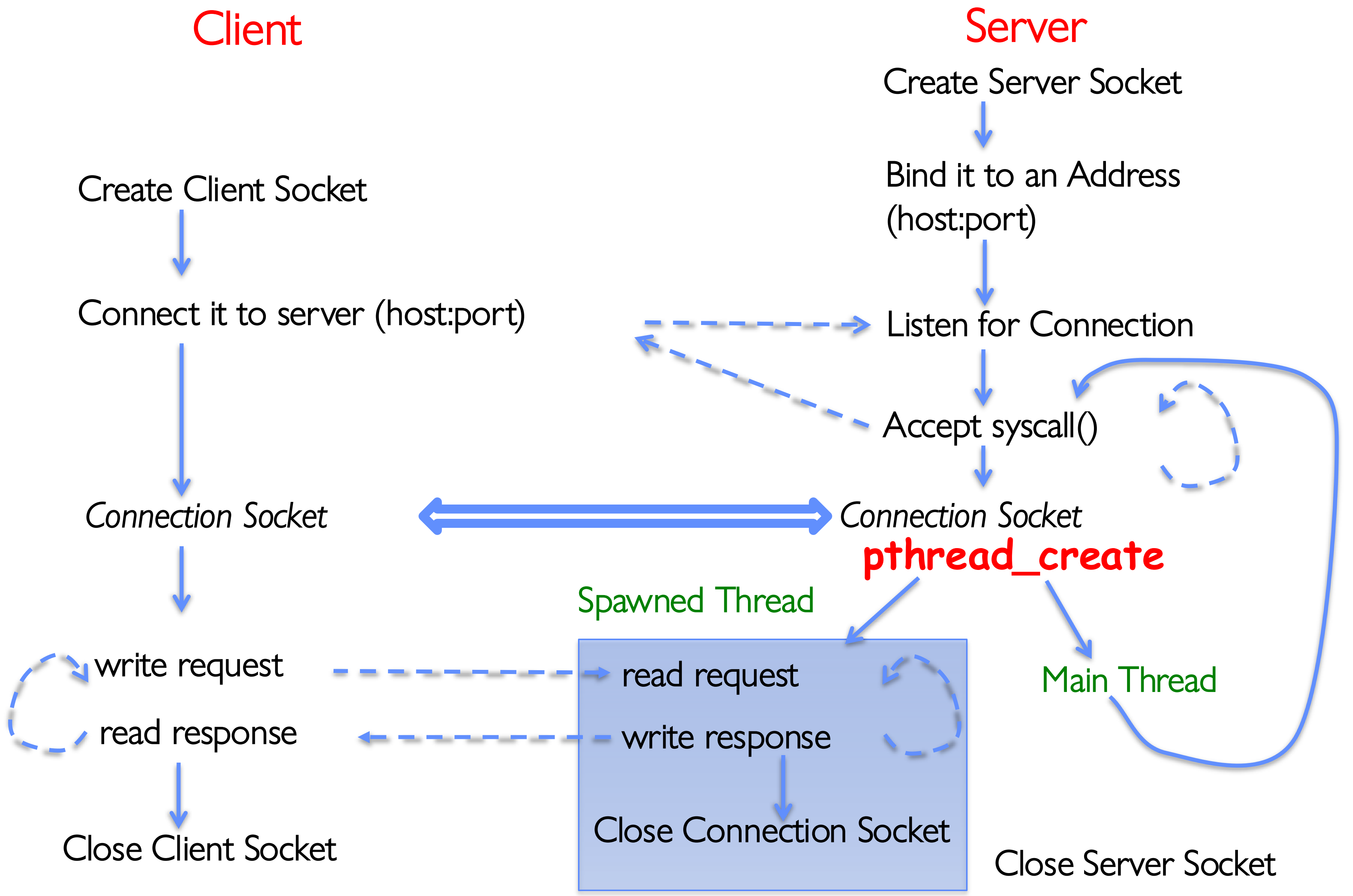
Server Code (v3)

```
// Socket setup code elided…
listen(server_socket, MAX_QUEUE);
while (1) {
  // Accept a new client connection, obtaining a new socket
  int conn_socket = accept(server_socket, NULL, NULL);
  pid_t pid = fork();
  if (pid == 0) {
    close(server_socket);
    serve_client(conn_socket);
    close(conn_socket);
    exit(0);
  } else {
    close(conn_socket);
    //wait(NULL);
  }
}
close(server_socket);
```

Faster Concurrent Server (without Protection)

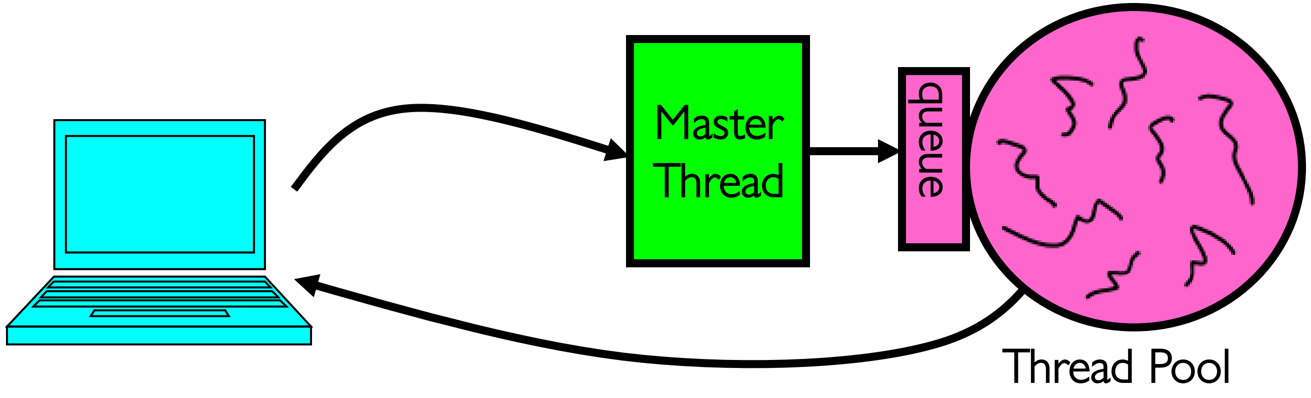
- Spawn a new *thread* to handle each connection
  - Lower overhead spawning process (less to do)
- Main *thread* initiates new client connections without waiting for previously spawned threads
- Why give up the protection of separate processes?
  - More efficient to create new threads
  - More efficient to switch between threads
- Even more potential for data races (need synchronization?)
  - Through shared memory structures
  - Through file system

Server with Concurrency, without Protection



Thread Pools: More Later!

- Problem with previous version: Unbounded Threads
  - When web-site becomes too popular – throughput sinks
- Instead, allocate a bounded "pool" of worker threads, representing the maximum level of multiprogramming




```
master() {
   allocThreads(worker,queue);
   while(TRUE) {
      con=AcceptCon();
      Enqueue(queue,con);
      wakeUp(queue);
   }
}
```

```
worker(queue) {
   while(TRUE) {
      con=Dequeue(queue);
      if (con==null)
         sleepOn(queue);
      else
         ServiceWebPage(con);
   }
}
```

Single-Process Pipe Example (not that interesting yet!)

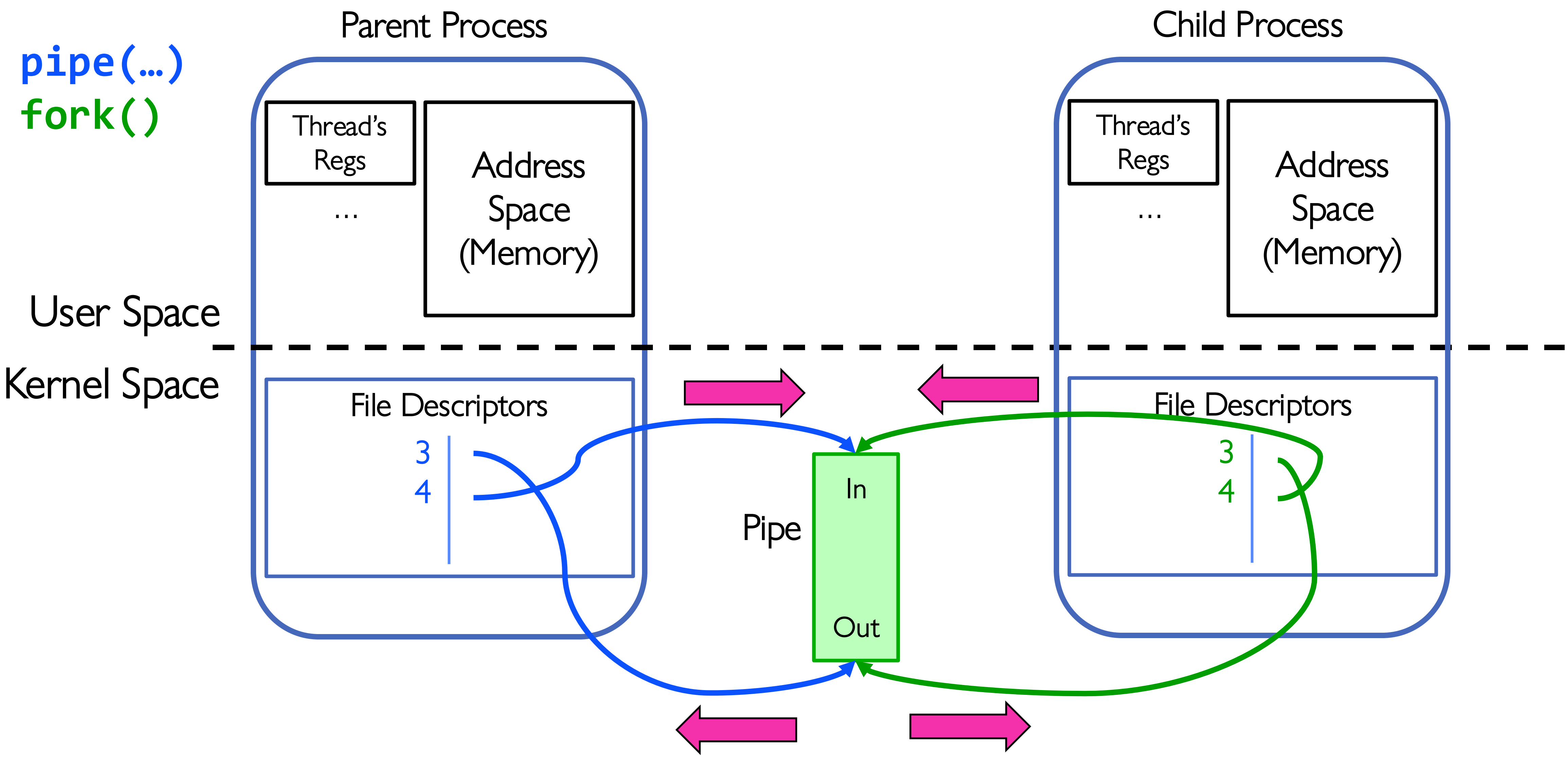
```c
#include <unistd.h>
int main(int argc, char *argv[])
{
  char *msg = "Message in a pipe.\n";
  char buf[BUFSIZE];
  int pipe_fd[2];
  if (pipe(pipe_fd) == -1) {
    fprintf (stderr, "Pipe failed.\n"); return EXIT_FAILURE;
  }
  ssize_t writelen = write(pipe_fd[1], msg, strlen(msg)+1);
  printf("Sent: %s [%ld, %ld]\n", msg, strlen(msg)+1, writelen);

  ssize_t readlen  = read(pipe_fd[0], buf, BUFSIZE);
  printf("Rcvd: %s [%ld]\n", msg, readlen);

  close(pipe_fd[0]);
  close(pipe_fd[1]);
}
```

Could be useful for multithreaded processes…

- Write to pipe_fd[1]
- Read from pipe_fd[0]

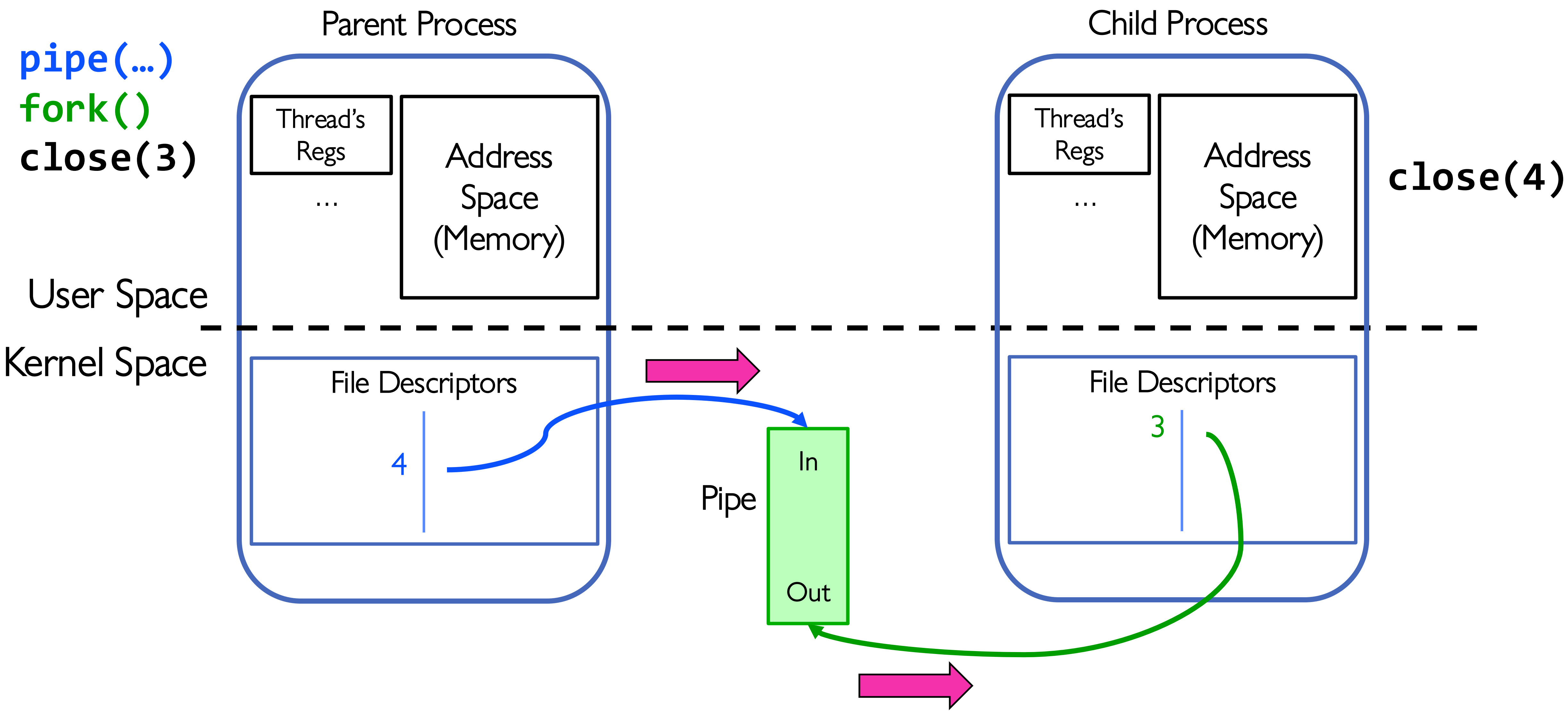
Example: Pipes *Between* Processes

```
pipe(…)
fork()
```



Example: Channel from Parent ⇒ Child

```
pipe(…)
fork()
close(3)
```



Inter-Process Communication (IPC): Parent $\Rightarrow$ Child

```c
// continuing from earlier
pid_t pid = fork();
if (pid < 0) {
  fprintf (stderr, "Fork failed.\n");
  return EXIT_FAILURE;
}
if (pid != 0) {
  close(pipe_fd[0]); // Not using this descriptor!
  ssize_t writelen = write(pipe_fd[1], msg, msglen);
  printf("Parent: %s [%ld, %ld]\n", msg, msglen, writelen);
} else {
  close(pipe_fd[1]); // Not using this descriptor!
  ssize_t readlen  = read(pipe_fd[0], buf, BUFSIZE);
  printf("Child Rcvd: %s [%ld]\n", msg, readlen);
}
```

Conclusion

- Recall: Everything is a file!
  - **`open()`**, **`read()`**, **`write()`**, and **`close()`** used for wide variety of I/O:
  - Devices (terminals, printers, etc.)
  - Regular files on disk
  - Networking (sockets)
  - Local interprocess communication (pipes, sockets)
- Processes have two parts
  - Threads (Concurrency)
  - Address Spaces (Protection)
- Various textbooks talk about *processes*
  - When this concerns concurrency, really talking about thread portion of a process
  - When this concerns protection, talking about address space portion of a process
- Stack is essential part of computation
  - Every thread has two stacks: user-level (in address space) and kernel
  - The kernel stack + support often called the "kernel thread"